

Mathematical Analysis

Mathematical Analysis

Author

Muho Choi

Email: sihyeon8240@gmail.com

GitHub: <https://github.com/sihyeon8240>

Source Code & Contributions

GitHub: <https://github.com/sihyeon8240/math>

Feel free to open issues or pull requests for corrections.

© 2026. Muho Choi.

This textbook is licensed under a [Creative Commons Attribution 4.0 International License](https://creativecommons.org/licenses/by/4.0/) (CC BY 4.0).

You are free to: Share, copy, and redistribute the material in any medium or format; adapt, remix, transform, and build upon the material for any purpose, even commercially.

Under the following terms: You must give appropriate credit, provide a link to the license, and indicate if changes were made.

Contents

Preface	v
1 Basic Topology	1
1.1 Metric Spaces	1
1.2 Compact Sets	23
1.3 Connected Sets	46
2 Sequences and Series	54
2.1 Convergent Sequences	54
2.2 Subsequences	60
2.3 Cauchy Sequences	60
2.4 Upper and Lower Limits	60
2.5 Some Special Sequences	60
2.6 Series	60
2.7 Series of Nonnegative Terms	60
2.8 The Number e	60
2.9 The Root and Ratio Tests	60
2.10 Power Series	60
2.11 Summation by Parts	60
2.12 Absolute Convergence	60
2.13 Addition and Multiplication of Series	60
2.14 Rearrangements	60
3 Continuity	61
3.1 Limits of Functions	61
3.2 Continuous Functions	61
3.3 Continuity and Compactness	61
3.4 Continuity and Connectedness	61
3.5 Discontinuities	61
3.6 Monotonic Functions	61
3.7 Infinite Limits and Limits at Infinity	61

4	Differentiation	62
4.1	The Derivative of a Real Function	62
4.2	Mean Value Theorems	62
4.3	The Continuity of Derivatives	62
4.4	L'Hospital's Rule	62
4.5	Derivatives of Higher Order	62
4.6	Taylor's Theorem	62
4.7	Differentiation of Vector-valued Functions	62
5	The Riemann-Stieltjes Integral	63
5.1	Definition and Existence of the Integral	63
5.2	Properties of the Integral	63
5.3	Integration and Differentiation	63
5.4	Integration of Vector-valued Functions	63
5.5	Rectifiable Curves	63
6	Sequences and Series of Functions	64
6.1	Discussion of Main Problem	64
6.2	Uniform Convergence	64
6.3	Uniform Convergence and Continuity	64
6.4	Uniform Convergence and Integration	64
6.5	Uniform Convergence and Differentiation	64
6.6	Equicontinuous Families of Functions	64
6.7	The Stone-Weierstrass Theorem	64
7	Some Special Functions	65
7.1	Power Series	65
7.2	The Exponential and Logarithmic Functions	65
7.3	The Trigonometric Functions	65
7.4	The Algebraic Completeness of the Complex Field	65
7.5	Fourier Series	65
7.6	The Gamma Function	65
8	Functions of Several Variables	66
8.1	Differentiation	66
8.2	The Contraction Principle	66
8.3	The Inverse Function Theorem	66
8.4	The Implicit Function Theorem	66
8.5	The Rank Theorem	66
8.6	Derivatives of Higher Order	66
8.7	Differentiation of Integrals	66

9	Integration of Differential Forms	67
9.1	Integration	67
9.2	Primitive Mappings	67
9.3	Partitions of Unity	67
9.4	Change of Variables	67
9.5	Differential Forms	67
9.6	Simplexes and Chains	67
9.7	Stokes' Theorem	67
9.8	Closed Forms and Exact Forms	67
9.9	Vector Analysis	67
10	The Lebesgue Theory	68
10.1	Set Functions	68
10.2	Construction of the Lebesgue Measure	68
10.3	Measure Spaces	68
10.4	Measurable Functions	68
10.5	Simple Functions	68
10.6	Integration	68
10.7	Comparison with the Riemann Integral	68
10.8	Integration of Complex Functions	68
10.9	Functions of Class $\mathcal{L}^2$	68

Preface

This textbook uses Lean 4 to formalize selected mathematical statements and proofs.

- **Tools.** Lean is an interactive theorem prover; Mathlib is a library of formalized mathematics. This edition uses Lean 4.34.0 and Mathlib v4.34.0 (revision [5ed296525643](#)).
- **Reading the proofs.** Each paired statement or proof step gives a mathematical explanation followed by its Lean code. The explanations can be read independently; code fragments are read in order within the complete declaration.
- **Verification.** Lean checks formal proofs under their stated assumptions. Verification covers only statements linked to checked declarations. Agreement between the explanations, displayed code, and checked proofs is reviewed separately.

Chapter 1

Basic Topology

1.1 Metric Spaces

Definition 1.1.1. A **metric** on X is a function $d : X \times X \rightarrow [0, \infty)$ such that, for all $p, q, z \in X$,

$$d(p, q) = 0 \iff p = q, \quad d(p, q) = d(q, p), \quad d(p, z) \leq d(p, q) + d(q, z).$$

The last condition is the **triangle inequality**. The pair (X, d) is a **metric space**.

Proposition 1.1.2. For $n \in \mathbb{N}$, the function $d(x, y) = \|x - y\|$ is a metric on $\mathbb{R}^n$.

```
theorem euclidean_metric (n : ℕ) :  
  (∀ x y : EuclideanSpace ℝ (Fin n), 0 ≤ ‖x - y‖) ∧  
  (∀ x y : EuclideanSpace ℝ (Fin n), ‖x - y‖ = 0 ↔ x = y) ∧  
  (∀ x y : EuclideanSpace ℝ (Fin n), ‖x - y‖ = ‖y - x‖) ∧  
  (∀ x y z : EuclideanSpace ℝ (Fin n),  
    ‖x - z‖ ≤ ‖x - y‖ + ‖y - z‖)
```

Proof. The norm is nonnegative. It vanishes exactly at the zero vector, so $\|x - y\| = 0$ precisely when $x = y$.

```
refine (fun x y => norm_nonneg _, ?_, ?_, ?_)  
  
· intro x y  
  rw [norm_eq_zero, sub_eq_zero]
```

Reversing a difference changes only its sign and leaves its norm unchanged.

```

· intro x y
  exact norm_sub_rev x y

```

Write $x - z = (x - y) + (y - z)$ and apply the Euclidean norm triangle inequality.

```

· intro x y z
  calc
    ||x - z|| = ||(x - y) + (y - z)|| := by
      rw [sub_add_sub_cancel]
    _ ≤ ||x - y|| + ||y - z|| := norm_add_le _ _

```

□

Euclidean spaces use this metric, including in dimension zero.

Definition 1.1.3. For $p \in X$ and $r > 0$, the **open ball** is

$$B(p, r) = \{q \in X : d(q, p) < r\}.$$

The **punctured ball** is $B(p, r) \setminus \{p\} = \{q \in X : 0 < d(q, p) < r\}$. A **neighborhood** of p is any set containing an open ball $B(p, r)$ with $r > 0$. A neighborhood need not itself be open or a ball.

Definition 1.1.4. A point p is a **limit point** of E if every positive-radius punctured ball about p meets E . The set of limit points, or **derived set**, is denoted by E' . A point is **interior** if some positive-radius ball about it lies in E ; the set of interior points is denoted by E° . An **isolated point** of E belongs to E and is its only point in some ball. We write $\text{Iso}(E)$ for the set of isolated points.

```

def isolatedPoints (E : Set X) : Set X :=
  {p | ∃ r : ℝ, 0 < r ∧ E ∩ ball p r = {p}}

```

Lemma 1.1.5. $p \in E'$ if and only if every $r > 0$ admits $q \in E$ with $q \neq p$ and $d(q, p) < r$.

```

theorem mem_derivedSet_iff (E : Set X) (p : X) :
  p ∈ derivedSet E ↔ ∀ r : ℝ, 0 < r → ∃ q ∈ E, q ≠ p ∧ dist q p < r

```

Proof. Every positive-radius ball is a neighborhood, so the neighborhood condition supplies a distinct point in that ball.

```

rw [mem_derivedSet, accPt_iff_nhds]
constructor

· intro h r hr
  obtain ⟨q, ⟨hq, hE⟩, hne⟩ := h (ball p r) (Metric.ball_mem_nhds p hr)
  exact ⟨q, hE, hne, hq⟩

```

Conversely, a neighborhood contains a positive-radius ball. A distinct point of the set in that ball also belongs to the neighborhood.

```

· intro h U hU
  obtain ⟨r, hr, hsub⟩ := Metric.mem_nhds_iff.mp hU
  obtain ⟨q, hE, hne, hq⟩ := h r hr
  exact ⟨q, ⟨hsub hq, hE⟩, hne⟩

```

□

Definition 1.1.6. A set E is **open** if every point of E is interior, and **closed** if its complement is open.

Definition 1.1.7. The **closure** $\overline{E}$ consists of points whose every positive-radius ball meets E .

Definition 1.1.8. A set E is **bounded** if there is a common upper bound for distances between its points. Equivalently, when X is nonempty, E lies in a positive-radius ball, as proved below. Subsets of the empty space are bounded. A set is **perfect** if it is closed and each of its points is a limit point. A set is **dense** if $\overline{E} = X$.

Lemma 1.1.9. E is bounded if and only if, whenever X is nonempty, some $p \in X$ and $r > 0$ satisfy $E \subseteq B(p, r)$.

```

theorem isBounded_iff (E : Set X) :
  Bornology.IsBounded E ↔ (Nonempty X → ∃ p : X, ∃ r > 0, E ⊆ ball p
    ↪ r)

```

Proof. Suppose all pairwise distances in E are at most C . If E is empty, any center in X and radius 1 suffice. Otherwise choose $p \in E$ and radius $\max(C, 0) + 1$.

```

rw [Metric.isBounded_iff]
constructor

· rintro (C, hC) hX
  rcases E.eq_empty_or_nonempty with rfl | (p, hp)

  · obtain (p) := hX
    exact (p, 1, zero_lt_one, empty_subset _)

  · refine (p, max C 0 + 1, by linarith [le_max_right C 0], ?_)
    intro q hq
    exact (hC hq hp).trans_lt (by linarith [le_max_left C 0])

```

Conversely, two points of a ball of radius r are at distance at most $2r$, by the triangle inequality. If X is empty there are no points to check, and zero is a bound.

```

· intro h
  by_cases hX : Nonempty X

  · obtain (p, r, _, hs) := h hX
    refine (r + r, ?_)
    intro x hx y hy
    have hx' : dist x p < r := hs hx
    have hy' : dist p y < r := by
      simp only [mem_ball, dist_comm] using hs hy
    exact (dist_triangle x p y).trans (add_le_add hx'.le hy'.le)

  · refine (0, ?_)
    intro x
    exact (hX (x)).elim

```

□

Proposition 1.1.10. *Every ball $B(p, r)$ is open. This also holds for nonpositive r , when the ball is empty.*

```

theorem ball_open (p : X) (r : ℝ) : IsOpen (ball p r)

```

Proof. Suppose $q \in B(p, r)$. The remaining distance $\delta = r - d(q, p)$ is positive.

```

apply Metric.isOpen_iff.mpr
intro q hq
refine (r - dist q p, sub_pos.mpr hq, ?_)

```

If $d(s, q) < \delta$, the triangle inequality gives $d(s, p) \leq d(s, q) + d(q, p) < r$. Thus the ball of radius δ about q stays inside $B(p, r)$.

```
intro s hs
change dist s p < r

have ht := dist_triangle s q p

change dist s q < r - dist q p at hs
linarith
```

□

Lemma 1.1.11. *For a finite index set S and positive numbers r_i for $i \in S$, there is $\delta > 0$ with $\delta \leq r_i$ for every $i \in S$.*

```
theorem finite_positive_lower_bound {ι : Type*} (s : Finset ι)
  (r : ι → ℝ) (hr : ∀ i ∈ s, 0 < r i) :
  ∃ δ > 0, ∀ i ∈ s, δ ≤ r i
```

Proof. Use induction on S . For the empty set, $\delta = 1$ works.

```
classical

induction s using Finset.induction_on with
| empty => exact ⟨1, zero_lt_one, by simp only [Finset.notMem_empty,
  ↪ false_implies, implies_true]⟩
```

On adjoining i , take the smaller of r_i and the positive bound already obtained for the remaining indices. This stays positive and is below every required radius.

```
| @insert i s hi ih =>
  obtain ⟨δ, hδ, hle⟩ := ih (fun j hj => hr j (Finset.mem_insert_of_mem
    ↪ hj))

  refine ⟨min (r i) δ, lt_min (hr i (Finset.mem_insert_self i s)) hδ,
    ↪ ?_⟩
  intro j hj
  rcases Finset.mem_insert.mp hj with rfl | hj

  · exact min_le_left _ _
  · exact (min_le_right _ _).trans (hle j hj)
```

□

Proposition 1.1.12. *If $p \in E'$ and $r > 0$, then $E \cap B(p, r)$ is infinite.*

```
theorem limit_point_infinite (E : Set X) (p : X)
  (hp : p ∈ derivedSet E) (r : ℝ) (hr : 0 < r) :
  (E ∩ ball p r).Infinite
```

Proof. Assume instead that this intersection is finite. After deleting p , its points still form a finite set, and all their distances from p are positive.

```
classical

intro hf

let s := (hf.subset (show (E ∩ ball p r) \ {p} ⊆ E ∩ ball p r from
  ↪ sdiff_subset)).toFinset

have hpos : ∀ q ∈ s, 0 < dist q p := by
  intro q hq

  have h := (Set.Finite.mem_toFinset _).mp hq

  exact dist_pos.mpr h.2
```

Choose a common positive lower bound δ for those distances. The limit-point condition supplies a distinct point of E within $\min(r, \delta)$ of p . It belongs to the finite set but has distance less than its lower bound, a contradiction.

```
obtain {δ, hδ, hle} := finite_positive_lower_bound s (fun q => dist q p)
  ↪ hpos
obtain {q, hq, hqp, hdist} := (mem_derivedSet_iff E p).mp hp (min r δ)
  ↪ (lt_min hr hδ)

have hqs : q ∈ s := (Set.Finite.mem_toFinset _).mpr
  (<{hq, hdist.trans_le (min_le_left _ _)}, hqp)

exact (not_lt_of_ge (hle q hqs)) (hdist.trans_le (min_le_right _ _))
```

□

Proposition 1.1.13. *E is open if and only if E^c contains all its limit points, that is, E^c is closed.*

```
theorem open_iff_compl_derivedSet (E : Set X) :
  IsOpen E ↔ derivedSet Ec ⊆ Ec
```

Proof. If E is open, a point in E has a ball contained in E . Such a ball prevents that point from being a limit point of the complement.

```
classical

constructor

· intro h p hp hpe

  obtain ⟨r, hr, hball⟩ := Metric.isOpen_iff.mp h p hpe
  obtain ⟨q, hq, _, hd⟩ := (mem_derivedSet_iff _ _).mp hp r hr

  exact hq (hball hd)
```

Conversely, suppose the complement contains its limit points. If $p \in E$ had no ball contained in E , each ball would contain a point outside E . That point differs from p , making p a limit point of the complement and contradicting $p \in E$.

```
· intro h
  apply Metric.isOpen_iff.mpr
  intro p hp
  by_contra hn

  have hl : p ∈ derivedSet Ec := by
    rw [mem_derivedSet_iff]
    intro r hr

    have hnsb : ¬ ball p r ⊆ E := fun hs => hn ⟨r, hr, hs⟩

    obtain ⟨q, hq, hqe⟩ := Set.not_subset.mp hnsb

    refine ⟨q, hqe, ?_, hq⟩
    intro heq
    exact hqe (heq.symm ▸ hp)

  exact h hl hp
```

□

Lemma 1.1.14. E is closed if and only if $E' \subseteq E$.

```
theorem closed_iff_derivedSet (E : Set X) :
  IsClosed E ↔ derivedSet E ⊆ E
```

Proof. A set is closed precisely when its complement is open. Apply the preceding result to E^c and take its complement twice.

```
have h := open_iff_compl_derivedSet Ec

rw [compl_compl] at h
exact isOpen_compl_iff.symm.trans h
```

□

Lemma 1.1.15. *E is perfect if and only if $E = E'$.*

```
theorem perfect_iff (E : Set X) : Perfect E ↔ E = derivedSet E
```

Proof. Closedness is equivalent to $E' \subseteq E$, by the preceding result. Every point being a limit point says $E \subseteq E'$. Together these are equality.

```
rw [perfect_def]
change (IsClosed E ∧ E ⊆ derivedSet E) ↔ E = derivedSet E
rw [closed_iff_derivedSet]
exact ⟨fun h => subset_antisymm h.2 h.1, fun h => ⟨h.symm.subset,
  ↪ h.subset⟩⟩
```

□

Proposition 1.1.16. *For any indexed family of open sets (E_i) , the union $\bigcup_i E_i$ is open.*

```
theorem open_union {ι : Type*} (E : ι → Set X)
  (h : ∀ i, IsOpen (E i)) : IsOpen (⋃ i, E i)
```

Proof. A point in the union belongs to some member E_i . Choose a ball contained in that member.

```
apply Metric.isOpen_iff.mpr
intro p hp

obtain ⟨i, hi⟩ := mem_iUnion.mp hp
obtain ⟨r, hr, hs⟩ := Metric.isOpen_iff.mp (h i) p hi
```

The same ball lies in the union. For an empty family there is no point to consider.

```
exact ⟨r, hr, fun q hq => mem_iUnion.mpr ⟨i, hs hq⟩⟩
```

□

Corollary 1.1.17. *For any indexed family of closed sets (E_i) , the intersection $\bigcap_i E_i$ is closed.*

```
theorem closed_intersection {ι : Type*} (E : ι → Set X)
  (h : ∀ i, IsClosed (E i)) : IsClosed (⋂ i, E i)
```

Proof. A limit point of the intersection is a limit point of each member: every nearby witness in the intersection belongs to that member. Each member is closed, so it contains the point; consequently the intersection contains it.

```
apply (closed_iff_derivedSet _).mpr
intro p hp
apply mem_iInter.mpr
intro i
apply (closed_iff_derivedSet _).mp (h i)
rw [mem_derivedSet_iff]
intro r hr

obtain ⟨q, hq, hn, hd⟩ := (mem_derivedSet_iff _ _).mp hp r hr

exact ⟨q, mem_iInter.mp hq i, hn, hd⟩
```

□

Lemma 1.1.18. *The intersection of two open sets is open.*

```
theorem open_intersection (E F : Set X) (hE : IsOpen E) (hF : IsOpen F)
  ↪ :
  IsOpen (E ∩ F)
```

Proof. At a point of the intersection, choose one positive radius for each set.

```

apply Metric.isOpen_iff.mpr
intro p hp

obtain (r, hr, hEr) := Metric.isOpen_iff.mp hE p hp.1
obtain (s, hs, hFs) := Metric.isOpen_iff.mp hF p hp.2

```

The smaller radius gives a ball lying in both sets and hence in their intersection.

```

refine (min r s, lt_min hr hs, ?_)
intro q hq
change dist q p < min r s at hq
exact (hEr (hq.trans_le (min_le_left _ _)), hFs (hq.trans_le
  ↪ (min_le_right _ _)))

```

□

Proposition 1.1.19. *A finite intersection of open sets is open. In particular this applies to*

$\bigcap_{j=0}^{n-1} E_j$ *for every* $n \in \mathbb{N}$.

```

theorem open_finite_intersection {ι : Type*} (s : Finset ι)
  (E : ι → Set X) (hE : ∀ i ∈ s, IsOpen (E i)) :
  IsOpen (⋂ i ∈ s, E i)

```

Proof. For no indices the intersection is X , which contains a unit ball about each of its points.

```

classical

induction s using Finset.induction_on with
| empty =>
  simp only [Finset.notMem_empty, iInter_of_empty, iInter_univ]
  exact Metric.isOpen_iff.mpr (fun p _ => ⟨1, zero_lt_one, subset_univ
    ↪ _⟩)

```

Adjoining an index intersects the already open finite intersection with one more open set. The preceding result completes the induction.

```

| @insert i s hi ih =>
  rw [Finset.set_biInter_insert]
  exact open_intersection _ _ (hE i (Finset.mem_insert_self i s))
    (ih (fun j hj => hE j (Finset.mem_insert_of_mem hj)))

```

□

Corollary 1.1.20. *A finite union of closed sets is closed, including the union over no indices.*

```
theorem closed_finite_union {ι : Type*} (s : Finset ι)
  (E : ι → Set X) (hE : ∀ i ∈ s, IsClosed (E i)) :
  IsClosed (⋃ i ∈ s, E i)
```

Proof. The complements are open, so their finite intersection is open.

```
have ho := open_finite_intersection s (fun i => (E i)c)
  (fun i hi => (hE i hi).isOpen_compl)
```

De Morgan's identity identifies this intersection with the complement of the desired union. The union is therefore closed.

```
have heq : (⋃ i ∈ s, E i)c = ⋂ i ∈ s, (E i)c := by
  ext p
  simp only [mem_compl_iff, mem_iUnion, mem_iInter, not_exists]

  exact isOpen_compl_iff.mp (heq ▸ ho)
```

□

Lemma 1.1.21. $\overline{E} = E \cup E'$, relating closure to the derived set.

```
theorem closure_eq_union_derivedSet (E : Set X) :
  closure E = E ∪ derivedSet E
```

Proof. If $p \in \overline{E}$, every ball about p meets E . Either $p \in E$, or each such witness differs from p , making p a limit point.

```
classical

ext p
constructor

· intro hp
  by_cases he : p ∈ E

  · exact Or.inl he
  · right
```

```

rw [mem_derivedSet_iff]
intro r hr

obtain ⟨q, hq, hd⟩ := Metric.mem_closure_iff.mp hp r hr

exact ⟨q, hq, fun h => he (h ▶ hq), by rwa [dist_comm]⟩

```

Conversely, a point of E itself witnesses that every ball meets E . For a limit point, use the distinct witness in each deleted ball. Both cases give membership in the closure.

```

· intro hp
  apply Metric.mem_closure_iff.mpr
  intro r hr
  rcases hp with hp | hp

· exact ⟨p, hp, by simp only [dist_self] using hr⟩

· obtain ⟨q, hq, _, hd⟩ := (mem_derivedSet_iff _ _).mp hp r hr

  exact ⟨q, hq, by rwa [dist_comm]⟩

```

□

Lemma 1.1.22. *Every set is contained in its closure.*

```

theorem subset_closure (E : Set X) : E ⊆ closure E

```

Proof. By the preceding characterization, every point of E belongs to the left member of the union $E \cup E'$.

```

rw [closure_eq_union_derivedSet]
exact fun _ hp => Or.inl hp

```

□

Lemma 1.1.23. *The closure of any set is closed.*

```

theorem closure_closed (E : Set X) : IsClosed (closure E)

```

Proof. Suppose p is a limit point of $\overline{E}$ and $r > 0$. Choose $q \in \overline{E}$ within $r/2$ of p , then $s \in E$ within $r/2$ of q .

```

apply (closed_iff_derivedSet _).mpr
intro p hp
apply Metric.mem_closure_iff.mpr
intro r hr

obtain ⟨q, hq, _, hdq⟩ := (mem_derivedSet_iff _ _).mp hp (r / 2)
  ↪ (half_pos hr)
obtain ⟨s, hs, hds⟩ := Metric.mem_closure_iff.mp hq (r / 2) (half_pos
  ↪ hr)

```

The triangle inequality puts s within r of p . Every ball about p therefore meets E , proving $p \in \overline{E}$.

```

refine ⟨s, hs, ?_⟩

have ht := dist_triangle p q s

rw [dist_comm q p] at hdq
linarith

```

□

Proposition 1.1.24. *For every $E \subseteq X$: $\overline{E}$ is closed; $E = \overline{E}$ if and only if E is closed; and every closed set F containing E also contains $\overline{E}$.*

```

theorem closure_properties (E : Set X) :
  IsClosed (closure E) ∧ (E = closure E ↔ IsClosed E) ∧
  (∀ F : Set X, IsClosed F → E ⊆ F → closure E ⊆ F)

```

Proof. We have just proved that $\overline{E}$ is closed. Therefore $E = \overline{E}$ implies that E is closed. Conversely, closedness gives $E' \subseteq E$, so $E \cup E' = E$.

```

refine ⟨closure_closed E, ?_, ?_⟩

· constructor
  · intro h
    rw [h]
    exact closure_closed E

· intro h
  rw [closure_eq_union_derivedSet]
  exact (union_eq_left.mpr ((closed_iff_derivedSet E).mp h)).symm

```

Let F be closed with $E \subseteq F$. A point of $\overline{E}$ either lies in E , hence in F , or is a limit point of E . In the latter case the same nearby witnesses make it a limit point of F , which belongs to F by closedness.

```

• intro F hF hEF p hp
  rw [closure_eq_union_derivedSet] at hp
  rcases hp with hp | hp

• exact hEF hp
• apply (closed_iff_derivedSet F).mp hF
  rw [mem_derivedSet_iff]
  intro r hr

  obtain ⟨q, hq, hn, hd⟩ := (mem_derivedSet_iff _ _).mp hp r hr

  exact ⟨q, hEF hq, hn, hd⟩

```

□

Proposition 1.1.25. *If $E \subseteq \mathbb{R}$ is nonempty and bounded above, then $\sup E \in \overline{E}$. If E is also closed, then $\sup E \in E$.*

```

theorem supremum_in_closure (E : Set ℝ) (hne : E.Nonempty) (hb :
  ↪ BddAbove E) :
  sSup E ∈ closure E ∧ (IsClosed E → sSup E ∈ E)

```

Proof. Write $c = \sup E$. For any $r > 0$, the smaller number $c - r$ is not an upper bound, so choose $q \in E$ with $c - r < q \leq c$. Then $|c - q| < r$, proving that every ball about c meets E .

```

have hc : sSup E ∈ closure E := by
  apply Metric.mem_closure_iff.mpr
  intro r hr

  obtain ⟨q, hq, hlt⟩ := exists_lt_of_lt_csSup hne (sub_lt_self (sSup E)
    ↪ hr)

  refine ⟨q, hq, ?_⟩
  rw [Real.dist_eq, abs_of_nonneg (sub_nonneg.mpr (le_csSup hb hq))]
  linarith

```

When E is closed, its closure equals E , so the established closure membership is membership in E itself.

```

exact ⟨hc, fun h => by
  have heq := (closure_properties E).2.1.mpr h

  rw [← heq] at hc
  exact hc)

```

□

Proposition 1.1.26. $\text{Iso}(E) = E \setminus E'$.

```

theorem isolated_eq_diff (E : Set X) : isolatedPoints E = E \ derivedSet
  ↪ E

```

Proof. If $E \cap B(p, r) = \{p\}$, then $p \in E$ and no distinct point of E is in that ball. This contradicts the limit-point condition, so $p \notin E'$.

```

classical

ext p
constructor

· rintro ⟨r, hr, heq⟩

  have hp : p ∈ E ∧ ball p r := heq.symm ▸ (mem_singleton p)

  refine ⟨hp.1, ?_⟩
  intro hl

  obtain ⟨q, hq, hn, hd⟩ := (mem_derivedSet_iff _ _).mp hl r hr

  have hmem : q ∈ E ∧ ball p r := ⟨hq, hd⟩

  rw [heq, mem_singleton_iff] at hmem
  exact hn hmem

```

Conversely, if $p \in E$ but $p \notin E'$, some positive-radius ball contains no distinct point of E . Its intersection with E is therefore exactly $\{p\}$.

```

· rintro ⟨hp, hn⟩

have hex : ∃ r > 0, ∀ q ∈ E, dist q p < r → q = p := by
  by_contra h
  push Not at h
  apply hn
  rw [mem_derivedSet_iff]
  intro r hr

  obtain ⟨q, hq, hd, hne⟩ := h r hr

  exact ⟨q, hq, hne, hd⟩

obtain ⟨r, hr, h⟩ := hex

refine ⟨r, hr, ?_⟩
ext q
constructor

· intro hq
  exact h q hq.1 hq.2

· intro hq
  rw [mem_singleton_iff] at hq
  subst q
  exact ⟨hp, by simp only [mem_ball, dist_self] using hr⟩

```

□

Lemma 1.1.27. $(E^\circ)^c = \overline{E^c}$.

```

theorem compl_interior_eq_closure_compl (E : Set X) :
  (interior E)c = closure Ec

```

Proof. Failure to be interior means that no positive-radius ball lies in E . Every such ball therefore contains a point of E^c , giving membership in its closure.

```

classical

ext p
rw [mem_compl_iff, mem_interior_iff_mem_nhds, Metric.mem_nhds_iff,
    Metric.mem_closure_iff]
constructor

```

```

· intro h r hr

  have hn : ¬ ball p r ⊆ E := fun hs => h ⟨r, hr, hs⟩

  obtain ⟨q, hq, hqe⟩ := Set.not_subset.mp hn

  exact ⟨q, hqe, by rwa [dist_comm]⟩

```

Conversely, if every ball meets E^c , no ball can be contained in E . Thus the point is not interior.

```

· intro h ⟨r, hr, hs⟩

  obtain ⟨q, hq, hd⟩ := h r hr

  exact hq (hs (by simp only [mem_ball, dist_comm] using hd))

```

□

Definition 1.1.28. The **boundary** is $\partial E = \overline{E} \setminus E^\circ$. The next lemma identifies it with $\overline{E} \cap \overline{E^c}$.

Lemma 1.1.29. $\partial E = \overline{E} \cap \overline{E^c}$.

```

theorem boundary_eq (E : Set X) :
  frontier E = closure E ∩ closure Ec

```

Proof. Write the difference as an intersection with the complement of the interior. The preceding lemma identifies this complement with the closure of the complement.

```

rw [frontier, sdiff_eq_compl_inter, compl_interior_eq_closure_compl,
  ↪ inter_comm]

```

□

Lemma 1.1.30. Any two points of $\mathbb{R}^0$ are equal, and every punctured ball in $\mathbb{R}^0$ is empty.

```

theorem zero_dimensional (p q : EuclideanSpace ℝ (Fin 0)) (r : ℝ) :
  p = q ∧ ball p r \ {p} = ∅

```

Proof. There are no coordinates on which two empty tuples can differ, so they are equal.

```

have heq :  $\forall$  x y : EuclideanSpace  $\mathbb{R}$  (Fin 0), x = y := by
  intro x y
  ext j
  exact Fin.elim0 j

```

Removing the center consequently removes every possible point of a ball.

```

refine (heq p q, ?_)
apply Set.eq_empty_iff_forall_notMem.mpr
intro x hx
exact hx.2 (heq x p)

```

□

Lemma 1.1.31. *For $n \in \mathbb{N}^+$, $p \in \mathbb{R}^n$, and $r > 0$, the punctured ball $B(p, r) \setminus \{p\}$ is nonempty.*

```

theorem euclidean_punctured_ball (n :  $\mathbb{N}^+$ )
  (p : EuclideanSpace  $\mathbb{R}$  (Fin n)) (r :  $\mathbb{R}$ ) (hr : 0 < r) :
   $\exists$  q : EuclideanSpace  $\mathbb{R}$  (Fin n), q  $\neq$  p  $\wedge$  dist q p < r

```

Proof. Since $n > 0$, the coordinate vector e_0 exists. Put $v = (r/2)e_0$, so $\|v\| = r/2 > 0$. Consequently $p + v$ differs from p .

```

let v : EuclideanSpace  $\mathbb{R}$  (Fin n) := EuclideanSpace.single (0, n.pos) (r
   $\hookrightarrow$  / 2)

have hv :  $\|v\| = r / 2$  := by
  rw [PiLp.norm_single, Real.norm_eq_abs, abs_of_pos (half_pos hr)]

refine (p + v, ?_, ?_)

· intro heq

  have hz : v = 0 := add_left_cancel (heq.trans (add_zero p).symm)

  rw [hz, norm_zero] at hv
  linarith

```

The distance from $p + v$ to p is $\|v\| = r/2 < r$. This is the required point in the punctured ball.

```

• rw [dist_eq_norm, add_sub_cancel_left, hv]
  linarith

```

□

Lemma 1.1.32. *If every positive-radius deleted ball in X is nonempty, then $\text{Iso}(E) \subseteq (E^c)'$.*

```

theorem isolated_limit_compl (E : Set X)
  (hX : ∀ p : X, ∀ r > 0, ∃ q : X, q ≠ p ∧ dist q p < r) :
  isolatedPoints E ⊆ derivedSet Ec

```

Proof. For an isolated point p , choose $r > 0$ with $E \cap B(p, r) = \{p\}$. For any $s > 0$, take a distinct point within $\min(r, s)$ of p .

```

intro p hp

obtain (r, hr, heq) := hp

rw [mem_derivedSet_iff]
intro s hs

obtain (q, hne, hd) := hX p (min r s) (lt_min hr hs)

```

If this point belonged to E , it would lie in $E \cap B(p, r)$ and equal p , a contradiction. It lies in E^c within radius s , witnessing the limit-point condition.

```

refine {q, ?_, hne, hd.trans_le (min_le_right _ _)}
intro hq

have hm : q ∈ E ∩ ball p r := {hq, hd.trans_le (min_le_left _ _)}

rw [heq, mem_singleton_iff] at hm
exact hne hm

```

□

Proposition 1.1.33. *For $n \in \mathbb{N}^+$ and $E \subseteq \mathbb{R}^n$, $\partial E = \text{Iso}(E) \cup \text{Iso}(E^c) \cup (E' \cap (E^c)')$.*

```

theorem euclidean_boundary (n : ℕ+)
  (E : Set (EuclideanSpace ℝ (Fin n))) :
  frontier E = isolatedPoints E ∪ isolatedPoints Ec ∪
    (derivedSet E ∩ derivedSet Ec)

```

Proof. An isolated point of either side is a limit point of the other, by the preceding results.

```

have hE := isolated_limit_compl E (euclidean_punctured_ball n)
have hEc := isolated_limit_compl Ec (euclidean_punctured_ball n)

rw [compl_compl] at hEc
rw [isolated_eq_diff] at hE hEc

```

Expand both closures in ∂E as the set together with its limit points.

```

rw [frontier, sdiff_eq_compl_inter,
  compl_interior_eq_closure_compl, closure_eq_union_derivedSet,
  closure_eq_union_derivedSet, isolated_eq_diff, isolated_eq_diff]

```

Points failing one limit-point condition are exactly the isolated points of one side. The remaining points form $E' \cap (E^c)'$.

```

ext p

have h1 := @hE p
have h2 := @hEc p

simp only [mem_inter_iff, mem_union, mem_sdiff, mem_compl_iff] at *
tauto

```

□

Proposition 1.1.34. *For $Y \subseteq X$, restricting d to $Y \times Y$ gives a metric on Y , with the same distances.*

```

theorem subspace_metric (Y : Set X) (p q z : Y) :
  dist p q = dist p.val q.val ∧ 0 ≤ dist p q ∧
  (dist p q = 0 ↔ p = q) ∧ dist p q = dist q p ∧
  dist p z ≤ dist p q + dist q z

```

Proof. Nonnegativity, symmetry, and the triangle inequality are inherited from X . Distance zero means equality of the underlying points, which is also equality as points of Y .

```
exact ⟨rfl, dist_nonneg, dist_eq_zero, dist_comm _ _, dist_triangle _ _
  ↪ _⟩
```

□

The resulting space is called the **subspace** (Y, d) . We continue to write $d(p, q)$ for its distance.

Definition 1.1.35. For $E \subseteq Y \subseteq X$, E is **open relative to Y** when it is open as a subset of the subspace Y . Equivalently, each $p \in E$ has $r > 0$ with $Y \cap B(p, r) \subseteq E$. It is **closed relative to Y** when it is closed in that subspace. The next lemma characterizes this by $E' \cap Y \subseteq E$.

Lemma 1.1.36. *For $E \subseteq Y$, E is closed in Y if and only if $E' \cap Y \subseteq E$.*

```
theorem relative_closed (E Y : Set X) (hEY : E ⊆ Y) :
  IsClosed ((Subtype.val : Y → X) ⁻¹' E) ↔ derivedSet E ∩ Y ⊆ E
```

Proof. A point of Y that is an ambient limit point of E is a subspace limit point: every witness lies in $E \subseteq Y$, and its distance is unchanged.

```
rw [closed_iff_derivedSet]
constructor
```

Conversely, each subspace witness has an underlying point in E with the same positive distance. Thus a subspace limit point is an ambient limit point lying in Y , and the stated condition puts it in E .

```
· intro h p hp
  apply h (show ((p, hp.2) : Y) ∈ derivedSet (Subtype.val ⁻¹' E) from
    ↪ ?_)
  rw [mem_derivedSet_iff]
  intro r hr

  obtain ⟨q, hq, hne, hd⟩ := (mem_derivedSet_iff _ _).mp hp.1 r hr

  refine ⟨⟨q, hEY hq⟩, hq, ?_, hd⟩
  intro heq
```

```

exact hne (congrArg Subtype.val heq)

· intro h p hp
  apply h
  refine {?_, p.property}
  rw [mem_derivedSet_iff]
  intro r hr

obtain ⟨q, hq, hne, hd⟩ := (mem_derivedSet_iff _ _).mp hp r hr

exact ⟨q.val, hq, fun heq => hne (Subtype.ext heq), hd⟩

```

□

Proposition 1.1.37. *For $E \subseteq Y \subseteq X$, E is open in Y if and only if $E = V \cap Y$ for some open $V \subseteq X$.*

```

theorem relative_open (E Y : Set X) (hEY : E ⊆ Y) :
  IsOpen ((Subtype.val : Y → X) ⁻¹' E) ↔
    ∃ V : Set X, IsOpen V ∧ E = V ∩ Y

```

Proof. Suppose E is open in Y . For each $p \in E$, choose $r_p > 0$ with $Y \cap B(p, r_p) \subseteq E$.

```

classical

constructor

· intro h

have hex : ∀ p : E, ∃ r > 0, Y ∩ ball p.val r ⊆ E := by
  intro p

obtain ⟨r, hr, hs⟩ := Metric.isOpen_iff.mp h ⟨p.val, hEY p.property⟩
  ↪ p.property

exact ⟨r, hr, fun q hq =>
  hs (show ((q, hq.1) : Y) ∈ ball ⟨p.val, hEY p.property⟩ r from
    ↪ hq.2)⟩

```

Let V be the union of these ambient balls. It is open. Each point of E lies in its own ball and in Y ; conversely, a point in $V \cap Y$ lies in one chosen ball, whose intersection with Y lies in E . Hence $E = V \cap Y$.

```

choose r hr hs using hex

refine (⋃ p : E, ball p.val (r p), open_union _ (fun p => ball_open _
  ↪ _), ?_)
ext q
constructor

· intro hq
  exact ⟨mem_iUnion.mpr ⟨⟨q, hq⟩, by simp only [mem_ball, dist_self]
    ↪ using hr ⟨q, hq⟩⟩, hEY hq⟩

· rintro ⟨hq, hy⟩

obtain ⟨p, hp⟩ := mem_iUnion.mp hq

exact hs p ⟨hy, hp⟩

```

Conversely, choose an ambient ball inside V about each point of $E = V \cap Y$. Restricting this ball to Y produces a subspace ball inside E .

```

· rintro ⟨V, hV, heq⟩
  apply Metric.isOpen_iff.mpr
  intro p hp

  have hpV : p.val ∈ V := (show p.val ∈ V ∩ Y from heq ▸ hp).1

  obtain ⟨r, hr, hs⟩ := Metric.isOpen_iff.mp hV p.val hpV

  refine ⟨r, hr, ?_⟩
  intro q hq
  change q.val ∈ E
  rw [heq]
  exact ⟨hs hq, q.property⟩

```

□

1.2 Compact Sets

Definition 1.2.1. A family $(G_i)_{i \in I}$ is an **open cover** of E if each G_i is open and $E \subseteq \bigcup_{i \in I} G_i$.

For $S \subseteq I$, the subfamily indexed by S is a **subcover** if $E \subseteq \bigcup_{i \in S} G_i$.

Definition 1.2.2. A set E is **compact** if every open cover admits a finite subcover.

Proposition 1.2.3. For $E \subseteq Y \subseteq X$, E is compact in the subspace Y if and only if it is compact in X .

```
theorem compact_subspace (E Y : Set X) (hEY : E ⊆ Y) :
  IsCompact ((Subtype.val : Y → X) ⁻¹' E) ↔ IsCompact E
```

Proof. First suppose E is compact in Y . Restrict an ambient open cover to Y . An ambient ball contained in a cover member restricts to a subspace ball, so each restricted member is open.

```
classical

constructor

· intro h
  apply isCompact_of_finite_subcover
  intro ι U hU hc

  have ho : ∀ i, IsOpen ((Subtype.val : Y → X) ⁻¹' U i) := by
    intro i
    apply Metric.isOpen_iff.mpr
    intro p hp

    obtain ⟨r, hr, hs⟩ := Metric.isOpen_iff.mp (hU i) p.val hp

    exact ⟨r, hr, fun q hq => hs hq⟩
```

The restricted family covers E in Y . A finite subcover there gives the same finite collection of ambient members covering E in X .

```
have hcov : (Subtype.val : Y → X) ⁻¹' E ⊆ ⋃ i, Subtype.val ⁻¹' U i :=
  ↪ by
    intro p hp

    obtain ⟨i, hi⟩ := mem_iUnion.mp (hc hp)

    exact mem_iUnion.mpr ⟨i, hi⟩

obtain ⟨s, hs⟩ := h.elim_finite_subcover _ ho hcov

refine ⟨s, ?_⟩
intro p hp
```

```

obtain ⟨i, hi⟩ := mem_iUnion.mp (hs (show ⟨p, hEY hp⟩ : Y) ∈
  ↪ Subtype.val ⁻¹' E from hp))
obtain ⟨his, hpi⟩ := mem_iUnion.mp hi

exact mem_iUnion.mpr ⟨i, mem_iUnion.mpr ⟨his, hpi⟩⟩

```

Conversely, let a family of subspace-open sets cover E . Each member is the intersection with Y of an ambient open set, by the relative-open-set proposition.

```

· intro h
  apply isCompact_of_finite_subcover
  intro ι U hU hc

  have hex : ∀ i, ∃ V : Set X, IsOpen V ∧ Subtype.val 'U i = V ∩ Y :=
    ↪ by
      intro i
      apply (relative_open _ Y (by
        rintro x ⟨p, _, rfl⟩
        exact p.property)).mp

      have heq : (Subtype.val : Y → X) ⁻¹' (Subtype.val 'U i) = U i :=
        preimage_image_eq _ Subtype.val_injective

      rw [heq]
      exact hU i

```

Choose those ambient open sets. They cover E in X , so compactness gives finitely many of them.

```

choose V hV heq using hex

have hcov : E ⊆ ⋃ i, V i := by
  intro p hp

  obtain ⟨i, hi⟩ := mem_iUnion.mp (hc (show ⟨p, hEY hp⟩ : Y) ∈
    ↪ Subtype.val ⁻¹' E from hp))

  have hm : p ∈ Subtype.val 'U i := ⟨⟨p, hEY hp⟩, hi, rfl⟩

  rw [heq i] at hm
  exact mem_iUnion.mpr ⟨i, hm.1⟩

```

For a point of $E \subseteq Y$, membership in a chosen ambient set is membership in its intersection with Y . Thus the corresponding finitely many original members cover E in the subspace.

```

obtain ⟨s, hs⟩ := h.elim_finite_subcover V hV hcov

refine ⟨s, ?_⟩
intro p hp

obtain ⟨i, hi⟩ := mem_iUnion.mp (hs hp)
obtain ⟨his, hpi⟩ := mem_iUnion.mp hi

have hm : p.val ∈ Subtype.val '' U i := (heq i).symm ▸ ⟨hpi, p.property⟩

obtain ⟨q, hq, he⟩ := hm

have hqp : q = p := Subtype.ext he

exact mem_iUnion.mpr ⟨i, mem_iUnion.mpr ⟨his, hqp ▸ hq⟩⟩

```

□

Proposition 1.2.4. *Every compact subset E of a metric space is closed.*

```

theorem compact_closed (E : Set X) (hE : IsCompact E) : IsClosed E

```

Proof. To prove that E^c is open, fix $p \notin E$. For each $q \in E$, let $r_q = d(q, p)/2 > 0$. The balls $B(q, r_q)$ cover E .

```

classical

apply isOpen_compl_iff.mp
apply Metric.isOpen_iff.mpr
intro p hp

let r : E → ℝ := fun q => dist q.val p / 2

have hr : ∀ q : E, 0 < r q := by
  intro q
  exact half_pos (dist_pos.mpr (fun heq => hp (heq ▸ q.property)))

have hcover : E ⊆ ⋃ q : E, ball q.val (r q) := by
  intro q hq
  exact mem_iUnion.mpr ⟨⟨q, hq⟩, by simp only [mem_ball, dist_self]
    ↪ using hr ⟨q, hq⟩⟩

```

Take a finite subcover and a common positive lower bound δ for its radii. This is also possible when the finite subcover is empty.

```

obtain ⟨s, hs⟩ := hE.elim_finite_subcover _ (fun q => ball_open _ _)
  ↪ hcover
obtain ⟨δ, hδ, hle⟩ := finite_positive_lower_bound s r (fun q _ => hr q)

refine ⟨δ, hδ, ?_⟩

```

If $x \in B(p, \delta) \cap E$, one of the selected balls contains x . Then $d(q, p) \leq d(q, x) + d(x, p) < r_q + \delta \leq 2r_q = d(q, p)$, a contradiction. Hence $B(p, \delta) \subseteq E^c$.

```

intro x hx hxe

obtain ⟨q, hq⟩ := mem_iUnion.mp (hs hxe)
obtain ⟨hqs, hxq⟩ := mem_iUnion.mp hq

have hd := dist_triangle q.val x p

rw [dist_comm q.val x] at hd

have hδq := hle q hqs

change dist x p < δ at hx
change dist x q.val < dist q.val p / 2 at hxq
change δ ≤ dist q.val p / 2 at hδq
linarith

```

□

Proposition 1.2.5. *Every compact subset of a metric space is bounded, with the stated empty-space convention.*

```

theorem compact_bounded (E : Set X) (hE : IsCompact E) :
  ↪ Bornology.IsBounded E

```

Proof. If X is nonempty, choose $p \in X$. The balls $B(p, n + 1)$, for $n \in \mathbb{N}$, cover E : every real distance is below some natural number by the Archimedean property.

```

classical

apply (isBounded_iff E).mpr
intro hX

obtain ⟨p⟩ := hX

```

```

have hc :  $E \subseteq \bigcup n : \mathbb{N}, \text{ball } p (n + 1 : \mathbb{R})$  := by
  intro q _

obtain (n, hn) := exists_nat_gt (dist q p)

exact mem_iUnion.mpr (n, by
  change dist q p < (n :  $\mathbb{R}$ ) + 1
  linarith)

```

Choose a finite subcover and set $R = 1 + \sum_{n \in S} (n + 1)$. This is positive, even when S is empty.

```

obtain {s, hs} := hE.elim_finite_subcover _ (fun n => ball_open _ _) hc

let R :  $\mathbb{R}$  := 1 +  $\sum n \in s, (n + 1 : \mathbb{R})$ 

have hnonneg :  $0 \leq \sum n \in s, (n + 1 : \mathbb{R})$  :=
  Finset.sum_nonneg (fun n _ => by positivity)

refine {p, R, by
  dsimp [R]
  linarith, ?_}

```

Each $q \in E$ belongs to a selected ball. Its radius is at most the sum of all selected radii, so $d(q, p) < R$. Thus $E \subseteq B(p, R)$.

```

intro q hq

obtain (n, hn) := mem_iUnion.mp (hs hq)
obtain (hns, hdist) := mem_iUnion.mp hn

have hle : (n + 1 :  $\mathbb{R}$ )  $\leq \sum k \in s, (k + 1 : \mathbb{R})$  :=
  Finset.single_le_sum (f := fun k :  $\mathbb{N}$  => (k + 1 :  $\mathbb{R}$ )) (fun k _ => by
    ↪ positivity) hns

change dist q p < R
change dist q p < (n + 1 :  $\mathbb{R}$ ) at hdist
dsimp [R]
linarith

```

□

Proposition 1.2.6. *If E is compact, F is closed, and $F \subseteq E$, then F is compact.*

```
theorem closed_subset_compact (E F : Set X) (hE : IsCompact E)
  (hF : IsClosed F) (hFE : F ⊆ E) : IsCompact F
```

Proof. Take an open cover of F . Add the open set F^c , giving the added member its own index distinct from the original indices.

```
classical

apply isCompact_of_finite_subcover
intro ι U hU hcover

let V : Option ι → Set X := fun i => match i with
  | none => Fc
  | some j => U j

have hV : ∀ i, IsOpen (V i) := by
  intro i
  cases i with
  | none => exact hF.isOpen_compl
  | some j => exact hU j
```

The enlarged family covers E : points in F are covered by the original family, and the others lie in F^c . Select a finite subcover of E .

```
have hc : E ⊆ ⋃ i, V i := by
  intro x _
  by_cases hx : x ∈ F

  · obtain ⟨i, hi⟩ := mem_iUnion.mp (hcover hx)

    exact mem_iUnion.mpr ⟨some i, hi⟩

  · exact mem_iUnion.mpr ⟨none, hx⟩

obtain ⟨s, hs⟩ := hE.elim_finite_subcover V hV hc
```

Discard the added member and keep the selected original indices. A point of F cannot lie in F^c , so it remains covered. This gives a finite subcover of F .

```

refine (s.biUnion Option.toFinset, ?_)
intro x hx

obtain (i, hi) := mem_iUnion.mp (hs (hFE hx))
obtain (his, hxi) := mem_iUnion.mp hi
cases i with
| none => exact False.elim (hxi hx)
| some j =>
  exact mem_iUnion.mpr (j, mem_iUnion.mpr
    (Finset.mem_biUnion.mpr
      (some j, his, by simp only [Option.toFinset_some,
        ↪ Finset.mem_singleton]), hxi))

```

□

Theorem 1.2.7. *If E is compact and $F \subseteq E$ is infinite, then $F' \cap E \neq \emptyset$.*

```

theorem infinite_subset_limit_point (E F : Set X) (hE : IsCompact E)
  (hFE : F ⊆ E) (hF : F.Infinite) : (derivedSet F ∩ E).Nonempty

```

Proof. Assume there is no such limit point. Then each $p \in E$ has a radius $r_p > 0$ for which the only possible point of F inside $B(p, r_p)$ is p itself. Otherwise every radius would have a distinct witness, making p a limit point.

```

classical

by_contra hn

have hlocal : ∀ p : E, ∃ r > 0, ∀ q ∈ F, dist q p.val < r → q = p.val :=
  ↪ by
    intro p
    by_contra h
    push Not at h
    apply hn
    refine (p.val, ?_, p.property)
    rw [mem_derivedSet_iff]
    intro r hr

    obtain (q, hq, hd, hne) := h r hr

    exact (q, hq, hne, hd)

```

These balls cover E , so choose finitely many of them.

```

choose r hr hsingle using hlocal

have hc : E ⊆ ⋃ p : E, ball p.val (r p) := by
  intro p hp
  exact mem_iUnion.mpr ((p, hp), by simp only [mem_ball, dist_self]
    ↪ using hr (p, hp))

obtain {s, hs} := hE.elim_finite_subcover _ (fun p => ball_open _ _) hc

```

Every point of F lies in one of the selected balls and must equal its center. Hence F is contained in the finite set of selected centers, contradicting its infinitude.

```

apply hF
apply (s.finite_toSet.image Subtype.val).subset
intro q hq

obtain {p, hp} := mem_iUnion.mp (hs (hFE hq))
obtain {hps, hd} := mem_iUnion.mp hp

exact {p, hps, (hsingle p q hq hd).symm}

```

□

Theorem 1.2.8. Suppose $(K_i)_{i \in I}$ are compact and every finite intersection $\bigcap_{i \in S} K_i$ is nonempty, including the intersection over no indices. Then $\bigcap_{i \in I} K_i$ is nonempty.

```

theorem compact_finite_intersection {ι : Type*} (K : ι → Set X)
  (hK : ∀ i, IsCompact (K i))
  (hfinite : ∀ s : Finset ι, (⋂ i ∈ s, K i).Nonempty) :
  (⋂ i, K i).Nonempty

```

Proof. First suppose I has an index i_0 . If the full intersection were empty, the complements of the K_i would cover K_{i_0} . These complements are open because compact sets are closed.

```

classical

by_cases hi : Nonempty ι

· obtain {i₀} := hi
  by_contra hn

```

```

have hc : K i0 ⊆ ⋃ i, (K i)c := by
  intro p _
  by_contra hp
  apply hn
  refine ⟨p, mem_iInter.mpr ?_⟩
  intro i
  by_contra hpi
  exact hp (mem_iUnion.mpr ⟨i, hpi⟩)

```

A finite subcover would make the intersection over its indices together with i_0 empty. The finite-intersection hypothesis instead supplies a point in all these sets, contradicting that subcover.

```

obtain ⟨s, hs⟩ := (hK i0).elim_finite_subcover _
  (fun i => (compact_closed _ (hK i)).is0open_compl) hc

obtain ⟨p, hp⟩ := hfinite (insert i0 s)

have hp0 := mem_iInter.mp (mem_iInter.mp hp i0) (Finset.mem_insert_self
  ↦ _ _)

obtain ⟨i, hi⟩ := mem_iUnion.mp (hs hp0)
obtain ⟨his, hpi⟩ := mem_iUnion.mp hi

exact hpi (mem_iInter.mp (mem_iInter.mp hp i) (Finset.mem_insert_of_mem
  ↦ his))

```

If I is empty, the hypothesis for the empty finite index set supplies a point of X . It automatically belongs to every member of the empty family.

```

• obtain ⟨p, _⟩ := hfinite ∅

  refine ⟨p, mem_iInter.mpr ?_⟩
  intro i
  exact False.elim (hi ⟨i⟩)

```

□

Lemma 1.2.9. *Let (a_n) and (b_n) be real sequences indexed by $n \in \mathbb{N}$ with $a_n \leq a_{n+1} \leq b_{n+1} \leq b_n$. There exists $x \in \mathbb{R}$ with $a_n \leq x \leq b_n$ for every n , so the nested intervals have nonempty intersection.*

```

theorem nested_intervals (a b :  $\mathbb{N} \rightarrow \mathbb{R}$ )
  (h :  $\forall n, a\ n \leq a\ (n + 1) \wedge a\ (n + 1) \leq b\ (n + 1) \wedge b\ (n + 1) \leq b\ n$ )
   $\hookrightarrow$  :
   $\exists x : \mathbb{R}, \forall n, a\ n \leq x \wedge x \leq b\ n$ 

```

Proof. The left endpoints increase and the right endpoints decrease. For any n, m , compare the indices: if $n \leq m$, then $a_n \leq a_m \leq b_m$; otherwise $a_n \leq b_n \leq b_m$. Thus each b_m bounds all left endpoints.

```

have ha : Monotone a := monotone_nat_of_le_succ (fun n => (h n).1)

have hb : Antitone b := antitone_nat_of_succ_le (fun n => (h n).2.2)

have hab :  $\forall n, a\ n \leq b\ n$  := fun n => (h n).1.trans ((h n).2.1.trans (h
 $\hookrightarrow$  n).2.2)

have hcross :  $\forall n\ m, a\ n \leq b\ m$  := by
  intro n m
  rcases le_total n m with hnm | hmn

  · exact (ha hnm).trans (hab m)
  · exact (hab n).trans (hb hmn)

```

The set of left endpoints contains a_0 and is bounded above by b_0 . Its supremum exists. Each a_n is below that supremum, and each b_n is an upper bound for it, giving a point in every interval.

```

have hne : (range a).Nonempty := ⟨a 0, mem_range_self 0⟩

have hbd : BddAbove (range a) := ⟨b 0, by
  rintro x ⟨n, rfl⟩
  exact hcross n 0⟩

refine ⟨sSup (range a), ?_⟩
intro n
exact ⟨le_csSup hbd (mem_range_self n), csSup_le hne (by
  rintro x ⟨m, rfl⟩
  exact hcross m n)⟩

```

□

Theorem 1.2.10. Let $k \in \mathbb{N}$ and let real endpoints satisfy $a_{n,j} \leq a_{n+1,j} \leq b_{n+1,j} \leq b_{n,j}$ for every $n \in \mathbb{N}$ and $0 \leq j < k$. Then the cells $\prod_{j=0}^{k-1} [a_{n,j}, b_{n,j}]$ have nonempty intersection.

```

theorem nested_cells (k : ℕ) (a b : ℕ → Fin k → ℝ)
  (h : ∀ n j, a n j ≤ a (n + 1) j ∧
    a (n + 1) j ≤ b (n + 1) j ∧ b (n + 1) j ≤ b n j) :
  ∃ x : EuclideanSpace ℝ (Fin k), ∀ n j, a n j ≤ x j ∧ x j ≤ b n j

```

Proof. In each coordinate apply the nested-interval result and choose a point belonging to every interval in that coordinate.

```

have hex : ∀ j, ∃ x : ℝ, ∀ n, a n j ≤ x ∧ x ≤ b n j :=
  fun j => nested_intervals (fun n => a n j) (fun n => b n j) (fun n =>
    ↪ h n j)

choose x hx using hex

```

The resulting vector belongs to every cell. For $k = 0$ the choice has no coordinates and produces the unique empty tuple.

```

exact ⟨WithLp.toLp 2 x, fun n j => hx j n⟩

```

□

We now prove compactness of cells from open covers. We first prove the interval case using a supremum, then combine finitely many coordinates. This also accounts for degenerate intervals and the zero-dimensional cell. The next supporting argument applies more generally to topological spaces: their open sets contain the empty and whole sets and are closed under arbitrary unions and finite intersections. Metric spaces have these properties by the preceding results. Compactness still means the finite-open-subcover property. A map is continuous in the open-set sense: inverse images of open sets are open. Only this defining property is needed.

Lemma 1.2.11. *For topological spaces A and B , if $K \subseteq A$ is compact and $f : A \rightarrow B$ is continuous, then $f(K)$ is compact.*

```

theorem compact_image {A B : Type*} [TopologicalSpace A]
  ↪ [TopologicalSpace B]
  (K : Set A) (hK : IsCompact K) (f : A → B) (hf : Continuous f) :
  IsCompact (f '' K)

```

Proof. Pull an open cover of $f(K)$ back along f . The inverse images are open and cover K , so finitely many cover it.

```

apply isCompact_of_finite_subcover
intro i U hU hc

have hcov :  $K \subseteq \bigcup i$ ,  $f^{-1} U i$  := by
  intro x hx

  obtain (i, hi) := mem_iUnion.mp (hc (x, hx, rfl))

  exact mem_iUnion.mpr (i, hi)

obtain (s, hs) := hK.elim_finite_subcover _ (fun i => (hU i).preimage
  ↪ hf) hcov

```

Every point of $f(K)$ has a preimage in K . That preimage belongs to a selected inverse image, so the point belongs to the corresponding selected cover member.

```

refine (s, ?_)
rintro y (x, hx, rfl)

obtain (i, hi) := mem_iUnion.mp (hs hx)
obtain (his, hxi) := mem_iUnion.mp hi

exact mem_iUnion.mpr (i, mem_iUnion.mpr (his, hxi))

```

□

Lemma 1.2.12. *A singleton in any metric space is compact.*

```

theorem singleton_compact {A : Type*} [MetricSpace A] (a : A) :
  IsCompact ({a} : Set A)

```

Proof. One member of any open cover contains the sole point of the singleton.

```

apply isCompact_of_finite_subcover
intro i U _ hc

obtain (i, hi) := mem_iUnion.mp (hc (mem_singleton a))

```

The subfamily containing only this member already covers the singleton, giving a finite subcover.

```

refine ({i}, ?_)
intro x hx
rw [mem_singleton_iff] at hx
subst x
exact mem_iUnion.mpr {i, mem_iUnion.mpr {Finset.mem_singleton_self _,
↪ hi}}

```

□

Lemma 1.2.13. *If $a, b \in \mathbb{R}$ and $a \leq b$, then $[a, b]$ is compact.*

```

theorem interval_compact (a b : ℝ) (hab : a ≤ b) : IsCompact (Icc a b)

```

Proof. Fix an open cover. Let T consist of the endpoints $x \in [a, b]$ for which $[a, x]$ has a finite subcover. A cover member containing a covers $[a, a]$, so $a \in T$.

```

classical

apply isCompact_of_finite_subcover
intro ι U hU hc

let T : Set ℝ := {x | x ∈ Icc a b ∧ ∃ s : Finset ι, Icc a x ⊆ ⋃ i ∈ s,
↪ U i}

obtain {i₀, hi₀} := mem_iUnion.mp (hc (show a ∈ Icc a b from {le_refl,
↪ hab}))

have ha : a ∈ T := by
  refine ({le_refl, hab}, {i₀}, ?_)
  intro x hx

  have heq : x = a := le_antisymm hx.2 hx.1

  subst x
  exact mem_iUnion.mpr {i₀, mem_iUnion.mpr {Finset.mem_singleton_self _,
↪ hi₀}}

```

The set T is nonempty and bounded above by b .

```

have hne : T.Nonempty := ⟨a, ha⟩
have hb : BddAbove T := ⟨b, fun x hx => hx.1.2⟩

```

The supremum $c = \sup T$ therefore exists, and $a \leq c \leq b$.

```

let c := sSup T
have hac : a ≤ c := le_csSup hb ha
have hcb : c ≤ b := csSup_le hne (fun x hx => hx.1.2)

```

Choose a cover member containing c . Since this member is open, some ball of radius $r > 0$ about c lies inside it.

```

obtain ⟨i, hi⟩ := mem_iUnion.mp (hc (show c ∈ Icc a b from ⟨hac, hcb⟩))
obtain ⟨r, hr, hball⟩ := Metric.isOpen_iff.mp (hU i) c hi

```

Choose $t \in T$ with $c - r/2 < t$. Set $d = \min(b, c + r/2)$, which is at least c . We will extend a finite cover of $[a, t]$ to $[a, d]$ by adjoining the chosen member.

```

obtain ⟨t, ht, hct⟩ := exists_lt_of_lt_csSup hne (show c - r / 2 < sSup
  ↪ T by
    dsimp [c]
    linarith)

obtain ⟨s, hs⟩ := ht.2

let d := min b (c + r / 2)
have hcd : c ≤ d := le_min hcb (by linarith)

have hd : d ∈ T := by
  refine ⟨⟨hac.trans hcd, min_le_left _ _⟩, insert i s, ?_⟩

```

A point $x \in [a, d]$ with $x \leq t$ is covered by the old subcover.

```

intro x hx
by_cases hxt : x ≤ t

• obtain ⟨j, hj⟩ := mem_iUnion.mp (hs ⟨hx.1, hxt⟩)

obtain ⟨hjs, hxj⟩ := mem_iUnion.mp hj

exact mem_iUnion.mpr ⟨j, mem_iUnion.mpr ⟨Finset.mem_insert_of_mem hjs,
  ↪ hxj⟩⟩

```

If $x > t$, then $c - r/2 < t < x \leq c + r/2$, so x is in the chosen ball. Thus the enlarged finite subcover covers $[a, d]$, proving $d \in T$.

```

• have hxu : x ≤ c + r / 2 := hx.2.trans (min_le_right _ _)

have hxball : x ∈ ball c r := by
  rw [mem_ball, Real.dist_eq, abs_lt]
  constructor <=> linarith

exact mem_iUnion.mpr (i, mem_iUnion.mpr (Finset.mem_insert_self _ _,
  ↪ hball hxball))

```

Since c is an upper bound of T , we have $d \leq c$. If $c < b$, both b and $c + r/2$ are greater than c , which would give $d > c$. Hence $c = b$ and $d = b$.

```

have hdc : d ≤ c := le_csSup hb hd

have hbc : b ≤ c := by
  by_contra h

have hlt : c < d := lt_min (lt_of_not_ge h) (by linarith)

exact (not_lt_of_ge hdc) hlt

have hdb : d = b := le_antisymm (min_le_left _ _) (hbc.trans hcd)

```

The finite subcover witnessing $d \in T$ now covers $[a, b]$.

```

rw [hdb] at hd
exact hd.2

```

□

Lemma 1.2.14. *For compact sets $K \subseteq A$ and $L \subseteq B$ in metric spaces, $K \times L$ is compact in the product metric $d((x, y), (x', y')) = \max(d(x, x'), d(y, y'))$.*

```

theorem compact_product {A B : Type*} [MetricSpace A] [MetricSpace B]
  (K : Set A) (L : Set B) (hK : IsCompact K) (hL : IsCompact L) :
  IsCompact (K ×s L)

```

Proof. Fix an open cover and $x \in K$. For each $y \in L$, choose a cover member and a product ball of radius $r_y > 0$ about (x, y) contained in it.

```

classical

apply isCompact_of_finite_subcover
intro ι U hU hc

have htube : ∀ x : K, ∃ δ > 0, ∃ s : Finset ι,
  ∀ x' ∈ ball x.val δ, ∀ y ∈ L, (x', y) ∈ ⋃ i ∈ s, U i := by
  intro x

have hlocal : ∀ y : L, ∃ i : ι, ∃ r > 0, ball (x.val, y.val) r ⊆ U i
  ↪ := by
  intro y

obtain ⟨i, hi⟩ := mem_iUnion.mp
  (hc (show (x.val, y.val) ∈ K ×s L from ⟨x.property, y.property⟩))
obtain ⟨r, hr, hs⟩ := Metric.isOpen_iff.mp (hU i) (x.val, y.val) hi

exact ⟨i, r, hr, hs⟩

```

The balls about the y coordinates cover L . Compactness selects finitely many; take a common positive lower bound δ for their radii.

```

choose i r hr hs using hlocal

have hcov : L ⊆ ⋃ y : L, ball y.val (r y) := by
  intro y hy
  exact mem_iUnion.mpr ⟨⟨y, hy⟩, by simp only [mem_ball, dist_self]
  ↪ using hr ⟨y, hy⟩

obtain ⟨t, ht⟩ := hL.elim_finite_subcover _ (fun y => ball_open _ _)
  ↪ hcov
obtain ⟨δ, hδ, hle⟩ := finite_positive_lower_bound t r (fun y _ => hr y)

```

If $d(x', x) < \delta$ and $y' \in L$, choose a selected ball containing y' . Both coordinate distances are below its radius, so (x', y') lies in its original cover member. Thus finitely many members cover the whole strip $B(x, \delta) \times L$.

```

refine ⟨δ, hδ, t.image i, ?_⟩
intro x' hx' y hy

obtain ⟨z, hz⟩ := mem_iUnion.mp (ht hy)
obtain ⟨hzt, hyz⟩ := mem_iUnion.mp hz

have hpair : (x', y) ∈ ball (x.val, z.val) (r z) := by
  change max (dist x' x.val) (dist y z.val) < r z
  exact max_lt (hx'.trans_le (hle z hzt)) hyz

```

```
exact mem_iUnion.mpr (i z, mem_iUnion.mpr (Finset.mem_image.mpr (z, hzt,
↪ rfl), hs z hpair))
```

Choose such a strip and finite index set for each $x \in K$. Their balls cover K , so finitely many strips suffice.

```
choose δ hδ s hs using htube

have hcov : K ⊆ ⋃ x : K, ball x.val (δ x) := by
  intro x hx
  exact mem_iUnion.mpr ((x, hx), by simp only [mem_ball, dist_self]
↪ using hδ (x, hx))

obtain (t, ht) := hK.elim_finite_subcover _ (fun x => ball_open _ _)
↪ hcov
```

Take the union of the finite index sets for those strips. For any $(x, y) \in K \times L$, one selected strip contains it, and its associated members cover it. The resulting finite family covers the product.

```
refine (t.biUnion s, ?_)
rintro (x, y) (hx, hy)

obtain (z, hz) := mem_iUnion.mp (ht hx)
obtain (hzt, hxz) := mem_iUnion.mp hz

obtain (i, hi) := mem_iUnion.mp (hs z x hxz y hy)
obtain (his, hxyi) := mem_iUnion.mp hi

exact mem_iUnion.mpr (i, mem_iUnion.mpr (Finset.mem_biUnion.mpr (z, hzt,
↪ his), hxyi))
```

□

Lemma 1.2.15. *For $n \in \mathbb{N}$ and real endpoints $a_j \leq b_j$ for $0 \leq j < n$, the coordinate box $\{x : a_j \leq x_j \leq b_j \text{ for all } j\}$ is compact in the finite-coordinate product metric.*

```
theorem pi_cell_compact (n : ℕ) (a b : Fin n → ℝ) (hab : ∀ j, a j ≤ b j)
↪ :
  IsCompact {x : Fin n → ℝ | ∀ j, a j ≤ x j ∧ x j ≤ b j}
```

Proof. Induct on n . With no coordinates, every vector is the same empty tuple, so the box is a compact singleton.

```

induction n with
| zero =>
  have heq : {x : Fin 0 → ℝ | ∀ j, a j ≤ x j ∧ x j ≤ b j} = {fun j =>
    ↪ Fin.elim0 j} := by
    ext x
    simp only [mem_ofPred_eq, mem_singleton_iff]
    constructor

    · intro _
      funext j
      exact Fin.elim0 j

    · intro _ j
      exact Fin.elim0 j

  rw [heq]
  exact singleton_compact _

```

For $n + 1$ coordinates, separate coordinate 0 from the remaining n . The map adjoining that first coordinate is continuous: each output-coordinate distance is bounded by the maximum product distance of the inputs, so $\delta = \varepsilon$ suffices. The interval for coordinate 0 and the remaining box are compact, so their product is compact.

```

| succ n ih =>
  let K := {x : Fin n → ℝ | ∀ j, a j.succ ≤ x j ∧ x j ≤ b j.succ}

  let f : ℝ × (Fin n → ℝ) → (Fin (n + 1) → ℝ) := fun p => Fin.cons p.1
    ↪ p.2

  have hf : Continuous f := by
    apply Metric.continuous_iff.mpr
    intro p ε hε
    refine ⟨ε, hε, ?_⟩
    intro q hq
    apply (dist_pi_lt_iff hε).mpr
    intro j

    have hpair : max (dist q.1 p.1) (dist q.2 p.2) < ε := hq

    refine Fin.cases ?_ (fun k => ?_) j

    · exact (max_lt_iff.mp hpair).1
    · exact (dist_le_pi_dist q.2 p.2 k).trans_lt (max_lt_iff.mp hpair).2

  have hk := compact_product (Icc (a 0) (b 0)) K (interval_compact _ _
    ↪ (hab 0))
    (ih (fun j => a j.succ) (fun j => b j.succ) (fun j => hab j.succ))

```

The image of this product is exactly the desired box: its first coordinate lies in the first interval, and the tail lies in the remaining intervals. Conversely, split any vector in the box into its first coordinate and tail. Compactness of continuous images completes the induction.

```

have heq : f '' (Icc (a 0) (b 0) xs K) =
  {x : Fin (n + 1) → ℝ | ∀ j, a j ≤ x j ∧ x j ≤ b j} := by
  ext x
  constructor

  · rintro ⟨⟨y, z⟩, ⟨hy, hz⟩, rfl⟩ j
    exact Fin.cases hy (fun k => hz k) j

  · intro hx
    refine ⟨(x 0, fun j => x j.succ), (hx 0, fun j => hx j.succ), ?_⟩
    funext j
    exact Fin.cases rfl (fun k => rfl) j

rw [← heq]
exact compact_image _ hk f hf

```

□

Lemma 1.2.16. *The coordinate identity from the finite-product metric on $\mathbb{R}^n$ to the Euclidean metric is continuous, including $n = 0$.*

```

theorem continuous_to_euclidean (n : ℕ) :
  Continuous (fun x : Fin n → ℝ => (WithLp.toLp 2 x : EuclideanSpace ℝ
    ↪ (Fin n)))

```

Proof. Fix $\varepsilon > 0$ and take $\delta = \varepsilon/(n + 1) > 0$. For two vectors within product distance δ , let D be that product distance and d their Euclidean distance.

```

apply Metric.continuous_iff.mpr
intro x ε hε

have hn : (0 : ℝ) ≤ n := Nat.cast_nonneg n

have hn1 : (0 : ℝ) < n + 1 := by
  positivity

refine (ε / (n + 1), div_pos hε hn1, ?_)
intro y hy

```

```

let D := dist y x

let d := dist (WithLp.toLp 2 y : EuclideanSpace ℝ (Fin n)) (WithLp.toLp
  ↪ 2 x)

have hD : 0 ≤ D := dist_nonneg

have hd : 0 ≤ d := dist_nonneg

```

Each coordinate distance is at most D . Squaring these nonnegative inequalities and summing the n terms gives $d^2 \leq nD^2$ by the Euclidean distance formula.

```

have hsquare : d ^ 2 ≤ (n : ℝ) * D ^ 2 := by
  rw [PiLp.dist_sq_eq_of_L2]
  calc
    ∑ j : Fin n, dist (y j) (x j) ^ 2 ≤ ∑ _j : Fin n, D ^ 2 := by
      apply Finset.sum_le_sum
      intro j _

    have hle : dist (y j) (x j) ≤ D := dist_le_pi_dist y x j

    have hnonneg : 0 ≤ dist (y j) (x j) := dist_nonneg

    nlinarith
  _ = (n : ℝ) * D ^ 2 := by
    simp only [Finset.sum_const, Finset.card_univ,
      Fintype.card_fin, nsmul_eq_mul]

```

Since $n \leq (n + 1)^2$ and both distances are nonnegative, $d \leq (n + 1)D$. The choice $D < \varepsilon/(n + 1)$ therefore gives $d < \varepsilon$. This estimate also covers the empty coordinate type.

```

have hbound : d ≤ ((n : ℝ) + 1) * D := by
  have hextra : 0 ≤ ((n : ℝ) ^ 2 + n + 1) * D ^ 2 :=
    mul_nonneg (by positivity) (sq_nonneg D)

  apply (sq_le_sq₀ hd (mul_nonneg hn1.le hD)).mp
  nlinarith only [hsquare, hextra]

have hsmall : ((n : ℝ) + 1) * D < ε := by
  have h := (lt_div_iff₀ hn1).mp hy

  nlinarith

exact hbound.trans_lt hsmall

```

□

Theorem 1.2.17. For every $n \in \mathbb{N}$ and real endpoints $a_j \leq b_j$, the cell $\prod_{j=0}^{n-1} [a_j, b_j]$ is compact in $\mathbb{R}^n$.

```
theorem cell_compact (n : ℕ) (a b : Fin n → ℝ) (hab : ∀ j, a j ≤ b j) :
  IsCompact {x : EuclideanSpace ℝ (Fin n) | ∀ j, a j ≤ x j ∧ x j ≤ b
    ↪ j}
```

Proof. The coordinate identity from the product description to the Euclidean description is continuous by the preceding distance estimate. Its image of the compact product box is compact.

```
have h := compact_image _ (pi_cell_compact n a b hab)
  (fun x : Fin n → ℝ => (WithLp.toLp 2 x : EuclideanSpace ℝ (Fin n)))
  (continuous_to_euclidean n)
```

This image is exactly the same coordinate box: a vector belongs to one precisely when its coordinates belong to the defining intervals. This also covers $n = 0$.

```
convert h using 1
ext x
constructor

· intro hx
  exact (WithLp.ofLp x, hx, rfl)

· rintro (y, hy, rfl)
  exact hy
```

□

Corollary 1.2.18. For $a, b \in \mathbb{R}$ with $a \leq b$, the closed interval $[a, b]$ is compact.

```
theorem closed_interval_compact (a b : ℝ) (hab : a ≤ b) : IsCompact (Icc
  ↪ a b)
```

Proof. This is the real interval result established in the construction of compact cells.

```
exact interval_compact a b hab
```

□

Theorem 1.2.19. For $n \in \mathbb{N}$ and $E \subseteq \mathbb{R}^n$, E is compact if and only if it is closed and bounded. This is the **Heine–Borel theorem**.

```
theorem heine_borel (n : ℕ) (E : Set (EuclideanSpace ℝ (Fin n))) :
  IsCompact E ↔ IsClosed E ∧ Bornology.IsBounded E
```

Proof. A compact set is closed and bounded by the metric-space results above. Conversely, suppose E is closed and bounded. Euclidean space has a zero vector even in dimension zero, so choose a ball $B(p, r)$ containing E .

```
constructor

· intro h
  exact ⟨compact_closed E h, compact_bounded E h⟩

· rintro ⟨hc, hb⟩

obtain ⟨p, r, hr, hs⟩ := (isBounded_iff E).mp hb ⟨0⟩
```

The box with coordinate intervals $[p_j - r, p_j + r]$ is compact. Each coordinate difference is bounded in absolute value by the Euclidean norm, so a point in $B(p, r)$ lies in this box. Hence E is a closed subset of a compact set and is compact.

```
let K : Set (EuclideanSpace ℝ (Fin n)) := {x | ∀ j, p j - r ≤ x j ∧ x j
  ↔ ≤ p j + r}

have hk : IsCompact K := cell_compact n _ _ (fun j => by linarith)

apply closed_subset_compact K E hk hc
intro x hx j

have hd : ‖x - p‖ < r := by
  simp only [mem_ball, dist_eq_norm] using hs hx

have hj : ‖(x - p) j‖ ≤ ‖x - p‖ := PiLp.norm_apply_le _ _

change ‖x j - p j‖ ≤ ‖x - p‖ at hj
rw [Real.norm_eq_abs] at hj

have hcoord := abs_le.mp (hj.trans (le_of_lt hd))

constructor <.> linarith
```

□

1.3 Connected Sets

Definition 1.3.1. Sets $A, B \subseteq X$ are **separated** if $\overline{A} \cap B = A \cap \overline{B} = \emptyset$.

```
def Separated {X : Type*} [MetricSpace X] (A B : Set X) : Prop :=
  Disjoint (closure A) B ∧ Disjoint A (closure B)
```

A set E is **connected** if it cannot be expressed as the union of two nonempty separated sets; otherwise it is **disconnected**. The empty set is connected under this convention. We therefore use Mathlib's `IsPreconnected E`; `IsConnected E` additionally requires E to be nonempty. The following lemma relates the standard predicate to the separation criterion.

Lemma 1.3.2. E is preconnected if and only if it has no decomposition into two nonempty separated sets.

```
theorem isPreconnected_iff {X : Type*} [MetricSpace X] (E : Set X) :
  IsPreconnected E ↔
    ∀ A B : Set X, A.Nonempty → B.Nonempty → E = A ∪ B → ¬ Separated A
    ↪ B
```

Proof. Use the closed-cover characterization of preconnectedness. If $E = A \cup B$ is a separation, the closed sets $\overline{A}$ and $\overline{B}$ cover E and both meet it. Preconnectedness supplies a point of E in both closures, contradicting separation whichever part contains it.

```
classical

rw [isPreconnected_closed_iff]
constructor

· intro h A B hA hB heq hsep
  have hcover : E ⊆ closure A ∪ closure B := by
    rw [heq]
    exact union_subset_union (subset_closure A) (subset_closure B)

  obtain ⟨a, ha⟩ := hA
  obtain ⟨b, hb⟩ := hB
  obtain ⟨x, hx, hxA, hxB⟩ := h (closure A) (closure B)
    (closure_closed A) (closure_closed B) hcover
  ⟨a, heq.symm ▸ Or.inl ha, subset_closure A ha⟩
  ⟨b, heq.symm ▸ Or.inr hb, subset_closure B hb⟩

  rw [heq] at hx
```

```

rcases hx with hx | hx

· exact Set.disjoint_left.mp hsep.2 hx hxB
· exact Set.disjoint_left.mp hsep.1 hxA hx

```

Conversely, let closed sets F, G cover E and both meet it. If their intersection misses E , then $E = (E \cap F) \cup (E \cap G)$. The closure of $E \cap F$ lies in F , and the closure of $E \cap G$ lies in G , by the previously proved closure property. Thus the two nonempty parts are separated, a contradiction.

```

· intro h F G hF hG hcover hEF hEG
  by_contra hn
  have heq : E = (E ∩ F) ∪ (E ∩ G) := by
    ext x
    exact ⟨fun hx => (hcover hx).elim (fun hF => Or.inl ⟨hx, hF⟩)
      (fun hG => Or.inr ⟨hx, hG⟩), fun hx => hx.elim And.left And.left⟩

  apply h (E ∩ F) (E ∩ G) hEF hEG heq
  constructor

  · apply Set.disjoint_left.mpr
    intro x hx hEG
    have hxF := (closure_properties (E ∩ F)).2.2 F hF inter_subset_right
      ↪ hx
    exact hn ⟨x, hEG.1, hxF, hEG.2⟩

  · apply Set.disjoint_left.mpr
    intro x hEF hx
    have hxG := (closure_properties (E ∩ G)).2.2 G hG inter_subset_right
      ↪ hx
    exact hn ⟨x, hEF.1, hEF.2, hxG⟩

```

□

Lemma 1.3.3. *If every point of $A \subseteq \mathbb{R}$ is at most c , the same is true for every point of $\overline{A}$.*

```

theorem closure_upper_bound (A : Set ℝ) (c : ℝ) (h : ∀ x ∈ A, x ≤ c) :
  ∀ x ∈ closure A, x ≤ c

```

Proof. If $x \in \overline{A}$ were greater than c , choose $y \in A$ within distance $x - c$ of x .

```

intro x hx
by_contra hn

have hc : c < x := lt_of_not_ge hn

obtain (y, hy, hd) := Metric.mem_closure_iff.mp hx (x - c) (sub_pos.mpr
  ↪ hc)

```

Then $|x - y| < x - c$ would imply $y > c$, contradicting the upper bound on A .

```

rw [Real.dist_eq, abs_lt] at hd

have hyc := h y hy

linarith

```

□

Lemma 1.3.4. *If every point of $A \subseteq \mathbb{R}$ is at least c , the same is true for every point of $\overline{A}$.*

```

theorem closure_lower_bound (A : Set ℝ) (c : ℝ) (h : ∀ x ∈ A, c ≤ x) :
  ∀ x ∈ closure A, c ≤ x

```

Proof. If $x \in \overline{A}$ were less than c , choose $y \in A$ within distance $c - x$ of x .

```

intro x hx
by_contra hn

have hc : x < c := lt_of_not_ge hn

obtain (y, hy, hd) := Metric.mem_closure_iff.mp hx (c - x) (sub_pos.mpr
  ↪ hc)

```

Then $|x - y| < c - x$ would imply $y < c$, contradicting the lower bound on A .

```

rw [Real.dist_eq, abs_lt] at hd

have hcy := h y hy

linarith

```

□

Lemma 1.3.5. *Suppose $E \subseteq \mathbb{R}$ contains every point strictly between any two of its elements. It is impossible to have $E = A \cup B$ with A, B separated and points $a \in A$, $b \in B$, $a < b$.*

```
theorem no_separation_of_between (E A B : Set ℝ)
  (hbetween : ∀ a ∈ E, ∀ b ∈ E, ∀ x : ℝ, a < x → x < b → x ∈ E)
  (heq : E = A ∪ B) (hsep : Separated A B)
  (a b : ℝ) (ha : a ∈ A) (hb : b ∈ B) (hab : a < b) : False
```

Proof. Let $S = A \cap [a, b]$. It contains a and is bounded above by b , so $c = \sup S$ exists and satisfies $a \leq c \leq b$. The supremum is in $\overline{S} \subseteq \overline{A}$.

```
let S := A ∩ Icc a b

have hane : a ∈ S := ⟨ha, le_refl, hab.le⟩

have hne : S.Nonempty := ⟨a, hane⟩

have hbd : BddAbove S := ⟨b, fun x hx => hx.2.2⟩

let c := sSup S

have hac : a ≤ c := le_csSup hbd hane

have hcb : c ≤ b := csSup_le hne (fun x hx => hx.2.2)

have hcS : c ∈ closure S := (supremum_in_closure S hne hbd).1

have hcA : c ∈ closure A := (closure_properties S).2.2 (closure A)
  (closure_closed A) (fun x hx => subset_closure A hx.1) hcS
```

Separation implies $c \notin B$, and thus $c < b$. If $c = a$, it is already in E ; otherwise it is strictly between a and b , so the between-points hypothesis puts it in E . Since $c \notin B$, we have $c \in A$.

```
have hcnotB : c ∉ B := fun h => Set.disjoint_left.mp hsep.1 hcA h

have hcltb : c < b := lt_of_le_of_ne hcb (fun h => hcnotB (h.symm ▸ hb))

have hcE : c ∈ E := by
  by_cases he : a = c
  · rw [← he, heq]
    exact Or.inl ha
```

```

· exact hbetween a (heq.symm ► Or.inl ha) b (heq.symm ► Or.inr hb) c
  (lt_of_le_of_ne hac he) hcltb

have hcain : c ∈ A := (show c ∈ A ∪ B from heq ► hcE).resolve_right
↪ hcnottB

```

Now separation gives $c \notin \overline{B}$. Consequently some $r > 0$ satisfies $d(c, y) \geq r$ for all $y \in B$.

```

have hcnottB : c ∉ closure B := fun h => Set.disjoint_left.mp hsep.2
↪ hcain h

have hex : ∃ r > 0, ∀ y ∈ B, r ≤ dist c y := by
  by_contra h
  push Not at h
  exact hcnottB (Metric.mem_closure_iff.mpr h)

obtain (r, hr, hfar) := hex

```

Choose $\delta = \min(r, b - c)/2$. It is positive, less than both r and $b - c$, and $x = c + \delta$ lies strictly between a and b . Hence $x \in E$.

```

let δ := min r (b - c) / 2

have hδ : 0 < δ := half_pos (lt_min hr (sub_pos.mpr hcltb))

have hδr : δ < r := (half_lt_self (lt_min hr (sub_pos.mpr
↪ hcltb))).trans_le (min_le_left _ _)

have hδb : δ < b - c := (half_lt_self (lt_min hr (sub_pos.mpr
↪ hcltb))).trans_le (min_le_right _ _)

let x := c + δ

have hax : a < x := by
  dsimp [x]
  linarith

have hxb : x < b := by
  dsimp [x]
  linarith

have hxE := hbetween a (heq.symm ► Or.inl ha) b (heq.symm ► Or.inr hb) x
↪ hax hxb

```

The distance $d(c, x) = \delta < r$ excludes x from B . Therefore $x \in A \cap [a, b] = S$, yet $x > c = \sup S$, a contradiction. Choosing half the available radius ensures that the new point is inside the excluded ball, not merely on its boundary.

```

have hxnotB :  $x \notin B$  := by
  intro hx

have hdist := hfar x hx

have hd :  $\text{dist } c \ x = \delta$  := by
  rw [Real.dist_eq, show  $c - x = -\delta$  by
    dsimp [x]
    ring, abs_neg, abs_of_pos hδ]

  rw [hd] at hdist
  linarith

have hxA := (show  $x \in A \cup B$  from heq  $\triangleright$  hxE).resolve_right hxnotB

have hxc :  $x \leq c$  := le_csSup hbd (show  $x \in S$  from {hxA, hax.le, hxb.le})
dsimp [x] at hxc
linarith

```

□

Theorem 1.3.6. A set $E \subseteq \mathbb{R}$ is connected if and only if, whenever $a, b \in E$ and $a < x < b$, one has $x \in E$.

```

theorem isPreconnected_iff_between (E : Set ℝ) :
  IsPreconnected E  $\leftrightarrow \forall a \in E, \forall b \in E, \forall x : \mathbb{R}, a < x \rightarrow x < b \rightarrow x \in E$ 

```

Proof. Suppose E is connected and a point x between $a, b \in E$ were missing. Split E into $A = E \cap (-\infty, x)$ and $B = E \cap (x, \infty)$. They cover E , and $a \in A, b \in B$ make both nonempty.

```

classical

rw [isPreconnected_iff]
constructor

· intro hc a ha b hb x hax hxb
  by_contra hx

let A := E ∩ Iio x

let B := E ∩ Ioi x

have heq : E = A ∪ B := by

```

```

ext y
constructor

· intro hy
  rcases lt_trichotomy y x with h | h | h

  · exact Or.inl ⟨hy, h⟩
  · exact False.elim (hx (h ▶ hy))
  · exact Or.inr ⟨hy, h⟩
· rintro (h | h) <=> exact h.1

```

The closure of A stays at or below x , so it misses B . Similarly the closure of B stays at or above x , so it misses A . This is a forbidden separation.

```

apply hc A B ⟨a, ha, hax⟩ ⟨b, hb, hxb⟩ heq
constructor

· apply Set.disjoint_left.mpr
  intro y hy hb

  have hle := closure_upper_bound A x (fun z hz => hz.2.le) y hy

  exact (not_lt_of_ge hle) hB.2

· apply Set.disjoint_left.mpr
  intro y hA hy

  have hle := closure_lower_bound B x (fun z hz => hz.2.le) y hy

  exact (not_lt_of_ge hle) hA.2

```

Conversely, suppose every intermediate point is in E and assume a separation $E = A \cup B$. Choose $a \in A$ and $b \in B$; separation makes them distinct. If $a < b$, the preceding supremum argument contradicts the separation. If $b < a$, interchange the two sets and apply the same argument.

```

· intro h A B hA hB heq hsep

obtain ⟨a, ha⟩ := hA
obtain ⟨b, hb⟩ := hB

have hne : a ≠ b := by
  intro he
  exact Set.disjoint_left.mp hsep.1 (subset_closure A ha) (he.symm ▶
    → hb)

rcases lt_or_gt_of_ne hne with hab | hba

```

```

· exact no_separation_of_between E A B h heq hsep a b ha hb hab

· have hsep' : Separated B A := ⟨hsep.2.symm, hsep.1.symm⟩

exact no_separation_of_between E B A h (heq.trans (union_comm _ _))
↪ hsep' b a hb ha hba

```

□

Chapter 2

Sequences and Series

2.1 Convergent Sequences

Sequences in this chapter are indexed by $\mathbb{N} = \{0, 1, 2, \dots\}$. The range $\{u_n : n \in \mathbb{N}\}$ records values; “finitely many exceptional terms” refers to indices, even when values repeat.

Definition 2.1.1. A **sequence** in X is a function $u : \mathbb{N} \rightarrow X$, with terms $u_n = u(n)$. It **converges to** $p \in X$ if for every $\varepsilon > 0$ there is $N \in \mathbb{N}$ such that $d(u_n, p) < \varepsilon$ whenever $n \geq N$. We write $u_n \rightarrow p$ or $\lim_{n \rightarrow \infty} u_n = p$. A sequence is **convergent** if it has a limit in its ambient space and **divergent** otherwise.

Definition 2.1.2. A sequence is **bounded** if its range $\{u_n : n \in \mathbb{N}\}$ is bounded.

Lemma 2.1.3. *If $u_n \rightarrow p$ in a metric space, then the range of u is bounded.*

```
theorem convergent_bounded (u : ℕ → X) (p : X) (h : Filter.Tendsto u
  ↪ Filter.atTop (nhds p)) :
  Bornology.IsBounded (range u)
```

Proof. Use convergence with $\varepsilon = 1$ to choose N beyond which every term is within 1 of p . Set $R = 1 + \sum_{n=0}^{N-1} d(u_n, p) > 0$.

```

rw [Metric.tendsto_atTop] at h
apply (isBounded_iff (range u)).mpr
intro _

obtain ⟨N, hN⟩ := h 1 zero_lt_one

let R := 1 + ∑ n ∈ Finset.range N, dist (u n) p

have hsum : 0 ≤ ∑ n ∈ Finset.range N, dist (u n) p :=
  Finset.sum_nonneg (fun n _ => dist_nonneg)

refine ⟨p, R, by
  dsimp [R]
  linarith, ?_⟩

```

For $n < N$, the distance $d(u_n, p)$ is at most the finite sum defining $R - 1$. For $n \geq N$, it is less than $1 \leq R$. Thus every term lies in $B(p, R)$. The argument still works if the initial segment is empty.

```

rintro x ⟨n, rfl⟩
change dist (u n) p < R
by_cases hn : n < N

· have hle := Finset.single_le_sum (f := fun n => dist (u n) p)
  (fun n _ => dist_nonneg) (Finset.mem_range.mpr hn)
  dsimp [R]
  linarith

· have htail := hN n (Nat.le_of_not_gt hn)
  dsimp [R]
  linarith

```

□

Lemma 2.1.4. *If a sequence converges to both p and q in a metric space, then $p = q$.*

```

theorem limit_unique (u : ℕ → X) (p q : X)
  (hp : Filter.Tendsto u Filter.atTop (nhds p))
  (hq : Filter.Tendsto u Filter.atTop (nhds q)) : p = q

```

Proof. If $p \neq q$, let $r = d(p, q) > 0$. Choose a convergence threshold for each limit with radius $r/2$.

```

rw [Metric.tendsto_atTop] at hp hq
by_contra hne

have hd : 0 < dist p q := dist_pos.mpr hne

obtain ⟨N, hN⟩ := hp (dist p q / 2) (half_pos hd)
obtain ⟨M, hM⟩ := hq (dist p q / 2) (half_pos hd)

```

Beyond the larger threshold, a single term is within $r/2$ of both points.

```

have hn := hN (max N M) (le_max_left _ _)

have hm := hM (max N M) (le_max_right _ _)

```

The triangle inequality would give $d(p, q) < r$, a contradiction. Thus the limits coincide.

```

have ht := dist_triangle p (u (max N M)) q

rw [dist_comm p (u (max N M))] at ht
linarith

```

□

Proposition 2.1.5. *A convergent sequence is bounded and has a unique limit.*

```

theorem convergent_properties (u : ℕ → X) (p : X) (hp : Filter.Tendsto u
  ↪ Filter.atTop (nhds p)) :
  Bornology.IsBounded (range u) ∧ ∀ q : X, Filter.Tendsto u
    ↪ Filter.atTop (nhds q) → p = q

```

Proof. The two preceding arguments give boundedness of the range and equality of any other limit with the given one.

```

exact ⟨convergent_bounded u p hp, fun q hq => limit_unique u p q hp hq⟩

```

□

Lemma 2.1.6. *For every $\varepsilon > 0$, some $N \in \mathbb{N}$ satisfies $1/(n+1) < \varepsilon$ for all $n \geq N$.*

```

theorem reciprocal_small (ε : ℝ) (hε : 0 < ε) :
  ∃ N : ℕ, ∀ n ≥ N, 1 / ((n : ℝ) + 1) < ε

```

Proof. By the Archimedean property, choose $N > 1/\varepsilon$.

```
obtain ⟨N, hN⟩ := exists_nat_gt (1 / ε)
```

For $n \geq N$, positivity gives $1 < N\varepsilon < (n+1)\varepsilon$. Dividing by $n+1 > 0$ proves the claim.

```
refine ⟨N, ?_⟩
intro n hn

have hcast : (N : ℝ) ≤ n := Nat.cast_le.mpr hn

have hprod : 1 < (N : ℝ) * ε := (div_lt_iff₀ hε).mp hn

apply (div_lt_iff₀ (show 0 < (n : ℝ) + 1 by positivity)).mpr
nlinarith
```

□

Example 2.1.7. Define $u_n = 1/(n+1)$ for $n \in \mathbb{N}$. Then $u_n \rightarrow 0$ in $\mathbb{R}$, but the same sequence diverges in the positive-real subspace $\mathbb{R}^+$. The shift makes every denominator positive under our zero-based indexing convention.

Choose the threshold from the preceding reciprocal estimate. Since each term is positive, its distance from zero is exactly $1/(n+1)$, which is eventually below ε .

If the sequence converged to a positive point p in the subspace, it would converge to the same real point because the distances are unchanged. Its real limit is already zero. Uniqueness would give $p = 0$, contradicting $p > 0$.

Lemma 2.1.8. $u_n \rightarrow p$ if and only if, for every $\varepsilon > 0$, the set of indices $\{n \in \mathbb{N} : d(u_n, p) \geq \varepsilon\}$ is finite.

```
theorem tendsto_iff_finite_exceptions (u : ℕ → X) (p : X) :
  Filter.Tendsto u Filter.atTop (nhds p) ↔ ∀ ε : ℝ, 0 < ε → {n : ℕ | ε
    ≤ dist (u n) p}.Finite
```

Proof. If $u_n \rightarrow p$, choose its threshold N . Every exceptional index is less than N , so the exception set is contained in a finite initial segment.

```
rw [Metric.tendsto_atTop]
constructor
```

Conversely, a finite set of natural indices has an upper bound N , including when it is empty. No index $n \geq N+1$ can be exceptional. Therefore every such term is within ε of p , proving convergence.

```

• intro h ε hε

obtain ⟨N, hN⟩ := h ε hε

apply (Finset.range N).finite_toSet.subset
intro n hn
apply Finset.mem_range.mpr
by_contra hnot
exact (not_lt_of_ge hn) (hN n (Nat.le_of_not_gt hnot))

• intro h ε hε

obtain ⟨N, hN⟩ := (h ε hε).bddAbove

refine ⟨N + 1, ?_⟩
intro n hn
by_contra hdist

have hle := hN (show n ∈ {n : ℕ | ε ≤ dist (u n) p} from le_of_not_gt
  ↪ hdist)

omega

```

□

Lemma 2.1.9. *If $p \in E'$, there is a sequence (u_n) of points in $E \setminus \{p\}$ that converges to p .*

```

theorem sequence_at_limit_point (E : Set X) (p : X) (hp : p ∈ derivedSet
  ↪ E) :
  ∃ u : ℕ → X, (∀ n, u n ∈ E ∧ u n ≠ p) ∧ Filter.Tendsto u
  ↪ Filter.atTop (nhds p)

```

Proof. For each $n \in \mathbb{N}$, use the limit-point condition to choose $u_n \in E$, distinct from p , with $d(u_n, p) < 1/(n + 1)$.

```

rw [mem_derivedSet_iff] at hp
have hex : ∀ n : ℕ, ∃ q ∈ E, q ≠ p ∧ dist q p < 1 / ((n : ℝ) + 1) :=
  fun n => hp _ (by positivity)

choose u hu hne hd using hex

refine ⟨u, fun n => ⟨hu n, hne n⟩, ?_⟩

```

Given $\varepsilon > 0$, the reciprocal estimate makes $1/(n + 1) < \varepsilon$ for all sufficiently large n . The chosen distance is smaller still, so $u_n \rightarrow p$.

```

rw [Metric.tendsto_atTop]
intro ε hε

obtain ⟨N, hN⟩ := reciprocal_small ε hε

exact ⟨N, fun n hn => (hd n).trans (hN n hn)⟩

```

□

Theorem 2.1.10. *A sequence converges to p exactly when every positive-radius ball $B(p, \varepsilon)$ contains all but finitely many of its terms, counted by indices. Also, if $p \in E'$, a sequence in $E \setminus \{p\}$ converges to p .*

```

theorem neighborhood_characterization (u : ℕ → X) (p : X) :
  (Filter.Tendsto u Filter.atTop (nhds p) ↔ ∀ ε : ℝ, 0 < ε → {n : ℕ |
    ε ≤ dist (u n) p}.Finite) ∧
  (∀ E : Set X, p ∈ derivedSet E →
    ∃ v : ℕ → X, (∀ n, v n ∈ E ∧ v n ≠ p) ∧ Filter.Tendsto v
    ↔ Filter.atTop (nhds p))

```

Proof. The finite-exception characterization proves the first assertion, including its converse. The construction from shrinking punctured balls proves the second assertion, with the terms explicitly required to belong to $E \setminus \{p\}$.

```

exact ⟨tendsto_iff_finite_exceptions u p, fun E hp =>
  ↔ sequence_at_limit_point E p hp⟩

```

□

2.2 Subsequences**2.3 Cauchy Sequences****2.4 Upper and Lower Limits****2.5 Some Special Sequences****2.6 Series****2.7 Series of Nonnegative Terms****2.8 The Number e** **2.9 The Root and Ratio Tests****2.10 Power Series****2.11 Summation by Parts****2.12 Absolute Convergence****2.13 Addition and Multiplication of Series****2.14 Rearrangements**

Chapter 3

Continuity

3.1 Limits of Functions

3.2 Continuous Functions

3.3 Continuity and Compactness

3.4 Continuity and Connectedness

3.5 Discontinuities

3.6 Monotonic Functions

3.7 Infinite Limits and Limits at Infinity

Chapter 4

Differentiation

4.1 The Derivative of a Real Function

4.2 Mean Value Theorems

4.3 The Continuity of Derivatives

4.4 L'Hospital's Rule

4.5 Derivatives of Higher Order

4.6 Taylor's Theorem

4.7 Differentiation of Vector-valued Functions

Chapter 5

The Riemann-Stieltjes Integral

5.1 Definition and Existence of the Integral

5.2 Properties of the Integral

5.3 Integration and Differentiation

5.4 Integration of Vector-valued Functions

5.5 Rectifiable Curves

Chapter 6

Sequences and Series of Functions

6.1 Discussion of Main Problem

6.2 Uniform Convergence

6.3 Uniform Convergence and Continuity

6.4 Uniform Convergence and Integration

6.5 Uniform Convergence and Differentiation

6.6 Equicontinuous Families of Functions

6.7 The Stone-Weierstrass Theorem

Chapter 7

Some Special Functions

7.1 Power Series

7.2 The Exponential and Logarithmic Functions

7.3 The Trigonometric Functions

7.4 The Algebraic Completeness of the Complex Field

7.5 Fourier Series

7.6 The Gamma Function

Chapter 8

Functions of Several Variables

8.1 Differentiation

8.2 The Contraction Principle

8.3 The Inverse Function Theorem

8.4 The Implicit Function Theorem

8.5 The Rank Theorem

8.6 Derivatives of Higher Order

8.7 Differentiation of Integrals

Chapter 9

Integration of Differential Forms

9.1 Integration

9.2 Primitive Mappings

9.3 Partitions of Unity

9.4 Change of Variables

9.5 Differential Forms

9.6 Simplexes and Chains

9.7 Stokes' Theorem

9.8 Closed Forms and Exact Forms

9.9 Vector Analysis

Chapter 10

The Lebesgue Theory

10.1 Set Functions

10.2 Construction of the Lebesgue Measure

10.3 Measure Spaces

10.4 Measurable Functions

10.5 Simple Functions

10.6 Integration

10.7 Comparison with the Riemann Integral

10.8 Integration of Complex Functions

10.9 Functions of Class $\mathcal{L}^2$

Bibliography

- [1] Walter Rudin, *Principles of Mathematical Analysis*, McGraw-Hill International Editions.
- [2] Kenneth Ross, *Elementary Analysis*, Springer Nature.
- [3] Steven Douglass, *Introduction to Mathematical Analysis*, Pearson.
- [4] Terence Tao, *Analysis I*, Springer Nature.